สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

การซ่อนข้อมูลและการฝังลายน้ำดิจิทัลในภาพด้วยการแยกค่าเอกฐาน (SVD)

SVD-based Digital Watermarking & Steganography

จัดทำโดย

[ชื่อ-นามสกุล] [รหัสนักศึกษา]

[ชื่อ-นามสกุล] [รหัสนักศึกษา]

[ชื่อ-นามสกุล] [รหัสนักศึกษา]

[ชื่อ-นามสกุล] [รหัสนักศึกษา]

เสนอ

[ชื่ออาจารย์ผู้สอน]

รายงานนี้เป็นส่วนหนึ่งของรายวิชาพีชคณิตเชิงเส้น (เวกเตอร์และเมทริกซ์) [รหัสวิชา]

ภาคเรียนที่ [ภาคเรียน/ปีการศึกษา]

สารบัญ

บทที่ 1 เป้าหมายและความน่าสนใจของแอปพลิเคชัน.....1

1.1 ที่มาและปัญหา.....1

1.2 ทำไมต้องเป็น SVD.....1

1.3 เป้าหมายของโครงงาน.....2

1.4 ความน่าสนใจ.....3

1.5 ภาพรวมของแอปพลิเคชัน.....4

บทที่ 2 ขั้นตอนการทำงานและการเตรียมข้อมูล.....5

2.1 ชุดข้อมูลและสถิติเบื้องต้น.....5

2.2 การ Clean ข้อมูล.....7

2.3 ผลก่อน/หลังการแก้ไขข้อมูล.....10

2.4 การแบ่งชุดข้อมูล.....11

บทที่ 3 การนำข้อมูลเข้าสู่โครงงาน: เวกเตอร์และเมทริกซ์.....12

3.1 ภาพเป็นเมทริกซ์.....12

3.2 แบ่งเป็นบล็อกสำหรับการซ่อนข้อความ.....12

3.3 ข้อมูลอื่นที่อยู่ในรูปเวกเตอร์/เมทริกซ์.....13

3.4 ค่าที่ใช้วัดผล (นิยามด้วยเวกเตอร์/เมทริกซ์).....14

บทที่ 4 ทฤษฎีเวกเตอร์และเมทริกซ์ที่ใช้ พร้อมตัวอย่างการคำนวณ 16

4.1 ทฤษฎีเวกเตอร์.....16

4.2 ทฤษฎีเมทริกซ์.....20

4.3 การประยุกต์ที่ 1: ซ่อนข้อความใน σ1 ของบล็อก (QIM).....26
4.4 การประยุกต์ที่ 2: ฝังลายน้ำ (Liu & Tan, 2002).....36
4.5 การประยุกต์ที่ 3: แก้บิตผิดด้วยเมทริกซ์ Hamming(7,4).....41
บทที่ 5 ขั้นตอนการประมวลผลที่วางแผนไว้และการเลือกพารามิเตอร์.....44
5.1 แผนการทำงานทั้งหมด.....44
5.2 การโจมตีที่ใช้ทดสอบ (กำหนดก่อนดูผล).....46
5.3 การเลือกพารามิเตอร์ (ชุด pilot 100 ภาพ).....46
บทที่ 6 ผลลัพธ์.....48
6.1 การซ่อนข้อความ: คุณภาพและความทนทาน.....48
6.2 ข้อความทั้งก้อนรอดไหม (Hamming เปิด/ปิด).....50
6.3 การฝังลายน้ำ.....51
6.4 ทฤษฎีเทียบกับผลจริง.....53
6.5 ความแข็งแรงของรหัสลับ.....54
6.6 ตรวจจับได้ไหมว่าภาพมีข้อมูลซ่อน.....55
6.7 แอปพลิเคชันเดโมและการทดสอบกับผู้ใช้.....57
บทที่ 7 ปัญหาที่พบระหว่างดำเนินโครงงานและวิธีแก้.....60
7.1 ข้อจำกัดที่ยังเหลืออยู่.....62
บทที่ 8 Source code และวิธีรัน.....64
บทที่ 9 สรุป.....67
บทที่ 10 แหล่งอ้างอิง.....69

บทที่ 1
เป้าหมายและความน่าสนใจของแอปพลิเคชัน

1.1 ที่มาและปัญหา

ภาพดิจิทัลถูกคัดลอกและส่งต่อได้ง่ายมาก ทำให้เกิดคำถามสองข้อที่พบบ่อยในชีวิตจริง ข้อแรกคือ **"ภาพนี้เป็นของใคร"** เจ้าของภาพต้องการหลักฐานความเป็นเจ้าของที่ติดไปกับตัวภาพ โดยไม่ทำให้ภาพดูเสีย เรียกว่า **การฝังลายน้ำดิจิทัล (digital watermarking)** ข้อที่สองคือ **"ส่งข้อความลับโดยไม่ให้ใครรู้ว่ามีข้อความได้ไหม"** คือซ่อนข้อความไว้ในภาพธรรมดา คนที่เห็นภาพไม่รู้ว่ามีข้อความ เรียกว่า **การซ่อนข้อมูล (steganography)**

ทั้งสองงานต้องการสิ่งเดียวกัน คือ "แก้ภาพเล็กน้อยจนตามองไม่ออก แต่แก้อย่างมีแบบแผนจนอ่านกลับได้" โครงงานนี้ใช้เครื่องมือจากพีชคณิตเชิงเส้นตัวเดียว คือ **การแยกค่าเอกฐาน (Singular Value Decomposition, SVD)** ทำทั้งสองงาน

1.2 ทำไมต้องเป็น SVD

ภาพระดับเทาคือเมทริกซ์ A ของค่าความสว่าง SVD เขียน A เป็นผลบวกของ "ชั้น" อันดับหนึ่ง (rank-1) ที่เรียงจากสำคัญมากไปน้อย

$$A = U\,\Sigma\,V^T = \sigma_1 u_1 v_1^T + \sigma_2 u_2 v_2^T + \cdots + \sigma_r u_r v_r^T,\ \sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0$$

สมบัติที่ทำให้ SVD เหมาะกับงานนี้มีสามข้อ (พิสูจน์และวัดจริงในบทที่ 4 และ 6) (1) ค่า σ_i แต่ละตัวบอก "ขนาด" ของชั้นนั้นได้ตรงๆ

และชั้นแรกเก็บพลังงานของภาพเฉลี่ย 88.9% (2) เพราะ U และ V เป็นเมทริกซ์เชิงตั้งฉาก การเปลี่ยน σ ไป d ทำให้ภาพเปลี่ยนด้วยขนาด (Frobenius norm) เท่ากับ d พอดี จึงควบคุมคุณภาพภาพได้แม่นยำ (3) ทฤษฎีบทของ Weyl รับประกันว่าถ้าภาพถูกรบกวนด้วย E ค่า σ จะขยับไม่เกิน $\|E\|_2$ จึงคาดการณ์ความทนทานได้ด้วยทฤษฎี

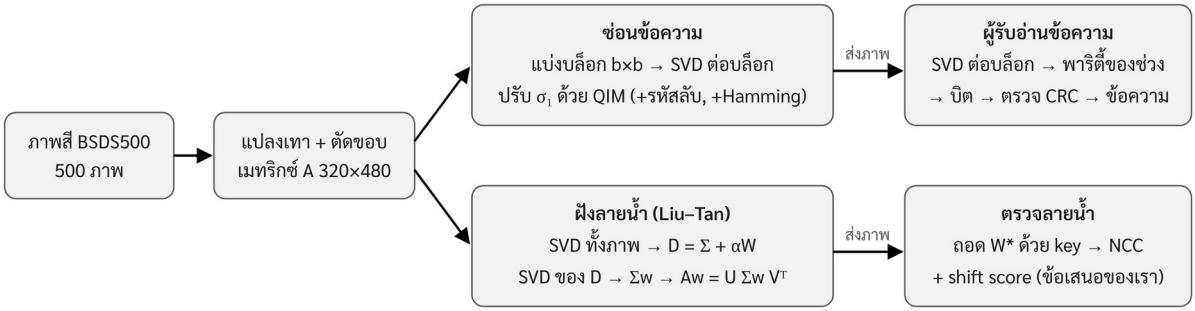
1.3 เป้าหมายของโครงงาน

1. สร้างระบบ **ซ่อนข้อความ** (ภาษาไทย/อังกฤษ) ในภาพด้วยการปรับค่า σ_1 ของบล็อก $b \times b$ แบบ Quantization Index Modulation (QIM) ให้อ่านกลับได้ถูกต้องทุกบิตเมื่อไม่ถูกโจมตี และภาพยังมีคุณภาพ PSNR ≥ 40 dB
2. สร้างระบบ **ฝังลายน้ำ** ตามวิธีของ Liu & Tan (2002) ที่ปรับค่า σ ของทั้งภาพ และตรวจสอบอย่างตรงไปตรงมาว่าการตรวจลายน้ำเชื่อถือได้แค่ไหน
3. วัดผลบนภาพจริง 400 ภาพ ภายใต้การโจมตีที่กำหนดไว้ล่วงหน้า (สัญญาณรบกวน, JPEG, การเจาะภาพ, การย่อขยาย, การเบลอ, การหมุน) และเทียบผลกับทฤษฎี
4. ทำแอปพลิเคชันเดโมที่ใช้งานได้จริงผ่านเบราว์เซอร์ ให้ผู้ใช้ซ่อน/อ่านข้อความ ฝัง/ถอดลายน้ำ และทดลองโจมตีเองได้

1.4 ความน่าสนใจ

- **ทฤษฎีเดียว ใช้ได้สองงาน:** เวกเตอร์ (ความยาว, ผลคูณภายใน, ความตั้งฉาก, cosine) และเมทริกซ์ (การคูณ, transpose, ค่าลักษณะเฉพาะ, SVD, norm) ถูกใช้จริงทุกขั้นตอน ไม่ใช่แค่ยกมาอ้าง
- **ผลที่วัดได้และอธิบายได้ด้วยทฤษฎี:** เช่น สูตรทำนายอัตราบิตผิด $2Q(\Delta/2s)$ ตรงกับค่าที่วัดได้ (ทำนาย 4.55% วัดได้ 4.67%) และการที่ตัวตรวจจับสังเกตเห็นการซ่อนข้อมูลได้ ก็อธิบายด้วยรูปร่างของเมทริกซ์ $u_1 v_1^T$
- **มีการประยุกต์ทฤษฎีเมทริกซ์บนตัวเลข 0/1:** รหัสแก้บิตผิด Hamming(7,4) คือการคูณเมทริกซ์แบบ mod 2 ทำให้ข้อความทนการบีบอัดภาพได้ดีขึ้น (โหมดบล็อกเล็ก + JPEG 75 รอด 60% → 97%)
- **ตรงไปตรงมาเรื่องข้อจำกัด:** รายงานนี้บอกทั้งสิ่งที่ทำได้และทำไม่ได้ เช่น ลายน้ำแบบ Liu–Tan ถอดได้จากภาพที่ไม่เคยฝัง (false positive) และการซ่อนข้อความไม่ทนการหมุนภาพ

1.5 ภาพรวมของแอปพลิเคชัน



รูปที่ 1.1 ภาพรวมการทำงานของแอปพลิเคชัน: ข้อมูลภาพผ่านการเตรียมเป็นเมทริกซ์ แล้วแยกเป็นสองงานที่ใช้ SVD เหมือนกัน

บทที่ 2
ขั้นตอนการทำงานและการเตรียมข้อมูล

2.1 ชุดข้อมูลและสถิติเบื้องต้น

โครงงานนี้ไม่ได้เก็บข้อมูลด้วย Google Form แต่ใช้ภาพจากชุดข้อมูลสาธารณะ **BSDS500** (Berkeley Segmentation Dataset 500; Arbeláez et al., 2011) ซึ่งเป็นภาพถ่ายธรรมชาติ คน สัตว์ และสิ่งก่อสร้าง ใช้กันแพร่หลายในงานประมวลผลภาพ **ข้อมูลหนึ่งชุด = ภาพหนึ่งภาพ** มีทั้งหมด **500 ชุด** (เกินขั้นต่ำ 150 ชุด) และใช้ครบทั้ง 500 ภาพ

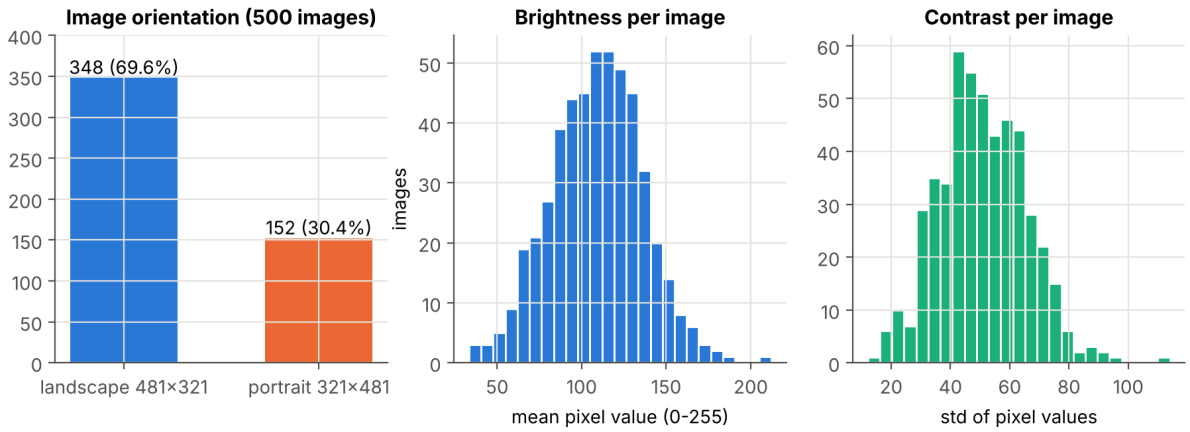
ตารางที่ 2.1 จำนวนและสัดส่วนของข้อมูลแต่ละรูปแบบ (ที่มา: `results/dataset_stats.csv`, notebook 00)

รายการ	จำนวน	สัดส่วน **(%)**
ภาพทั้งหมด	500	100.0
กลุ่ม train / val / test (การแบ่งของ BSDS500)	200 / 100 / 200	40.0 / 20.0 / 40.0
รูปแบบไฟล์ JPEG	500	100.0
โหมดสี RGB (3 ช่องสี)	500	100.0
ภาพแนวนอน 481×321 พิกเซล	348	69.6
ภาพแนวตั้ง 321×481 พิกเซล	152	30.4

รายการ	จำนวน	สัดส่วน **(%)**
ภาพที่ด้านกว้างและสูงหารด้วย 8 ลงตัว	0	0.0

ตารางที่ 2.2 สถิติของค่าพิกเซลและค่าเอกฐาน (คำนวณหลังแปลงเป็นภาพเทา) คอลัมน์ "ค่าสูงสุด" คือภาพที่มีค่านั้นสูงที่สุดในชุดข้อมูล เช่น ภาพที่สว่างที่สุดมีความสว่างเฉลี่ย 212.1 (ที่มา: `results/clean_manifest.csv`, notebook 01)

ค่าที่วัดต่อภาพ	ค่าเฉลี่ย	ค่าสูงสุด	ค่าต่ำสุด
ขนาดไฟล์ (KB)	72.25	123.10	23.10
ความสว่างเฉลี่ยของภาพ (0–255)	109.0	212.1	33.5
ความต่างแสง (ส่วนเบี่ยงเบนมาตรฐานของพิกเซล)	50.7	114.1	12.1
พิกเซลที่มืดที่สุดในภาพ	11.7	55.0	0.0
พิกเซลที่สว่างที่สุดในภาพ	250.2	255.0	166.0
สัดส่วนพลังงานของ σ_1 ทั้งภาพ (%)	88.9	99.8	59.6



รูปที่ 2.1 การกระจายของข้อมูล: สัดส่วนภาพแนวนอน/แนวตั้ง, ความสว่างเฉลี่ยต่อภาพ และความต่างแสงต่อภาพ

ตารางที่ 2.3 พิกเซลที่ชนขอบ 0 หรือ 255 แก้ต่อไม่ได้ (ตัดทิ้งส่วนเกิน) ข้อมูลที่ซ่อนจึงเสี่ยงเสีย ปัญหานี้ทำให้เราต้องเพิ่ม "ระยะเผื่อ" ในการซ่อน (ข้อ 4.3.4 และบทที่ 7)

ความเสี่ยงเรื่องพิกเซลชนขอบ (0 หรือ 255)	จำนวนภาพ	สัดส่วน (%)
มีพิกเซลขาวสุด (255) อย่างน้อย 1 จุด	207	41.4
มีพิกเซลดำสุด (0) อย่างน้อย 1 จุด	76	15.2
มีพิกเซลชนขอบมากกว่า 1% ของภาพ	44	8.8
มีพิกเซลชนขอบมากกว่า 5% ของภาพ	19	3.8

2.2 การ Clean ข้อมูล

ทุกภาพถูกตรวจสี่ข้อก่อนใช้ เกณฑ์ตั้งไว้ก่อนดูข้อมูล (ค่า 10 และ 64 เป็นค่าที่เรากำหนดเอง)

ตารางที่ 2.4 ผลการตรวจคุณภาพข้อมูล (ภาพที่เรียบที่สุดคือ 80099.jpg ส่วนเบี่ยงเบน 12.1 ยังผ่านเกณฑ์)

การตรวจ	สิ่งที่จับ	เกณฑ์	พบ (ภาพ)	%
เปิดไม่ได้	ไฟล์เสียหรือไม่ครบ	เปิดด้วย Pillow ไม่ได้	0	0.0
ภาพซ้ำ	ภาพเดียวกันสองครั้ง (ทำให้ผลเกินจริง)	MD5 ของพิกเซลตรงกับภาพก่อนหน้า	0	0.0
ภาพเรียบเกินไป	แทบไม่มีรายละเอียด	ส่วนเบี่ยงเบนมาตรฐาน < 10	0	0.0
เล็กเกินไป	มีบล็อกไม่พอซ่อนข้อมูล	ด้านใดด้านหนึ่ง < 64 พิกเซล	0	0.0

ตารางที่ 2.5 สัดส่วนข้อมูลที่สมบูรณ์ / ต้องแก้ไข / ต้องตัดทิ้ง ทุกภาพต้องแก้ไข เพราะเป็นภาพสีและขนาด 481 หรือ 321 ไม่หารด้วย 8 หรือ 16 ลงตัว

สถานะของข้อมูล	ความหมาย	จำนวน	สัดส่วน (%)
สมบูรณ์ ใช้ได้ทันที	ไม่ต้องแก้อะไร	0	0.0

สถานะของข้อมูล	ความหมาย	จำนวน	สัดส่วน (%)
ต้องแก้ไข	แปลงเป็นภาพเทา และตัดขอบให้ด้านหาร 16 ลงตัว	500	100.0
ไม่สมบูรณ์ ต้องตัดทิ้ง	ไม่ผ่านการตรวจในตารางที่ 2.4	0	0.0

การแก้ไขมีสองขั้น (โค้ดใน `src/svdmark/data.py`)

1. **แปลงเป็นภาพเทา** ด้วยสูตรความสว่าง ITU-R BT.601: $Y = 0.299\,R + 0.587\,G + 0.114\,B$ ตัวอย่างพิกเซล $(R, G, B) = (200, 100, 50)$ ได้ $Y = 59.8 + 58.7 + 5.7 = 124.2 \to 124$ (Pillow ปัดเป็นจำนวนเต็ม)
2. **ตัดขอบตรงกลาง** ให้ทั้งสองด้านหาร 16 ลงตัว เพื่อให้แบ่งเป็นบล็อก 4×4, 8×8 และ 16×16 ได้พอดี: $481 \to 480$, $321 \to 320$ เศษที่เหลือ 1 แถว/1 คอลัมน์ ถูกตัดทิ้ง เลือก "ตัด" แทน "ย่อขยาย" เพราะการย่อขยายจะคำนวณค่าพิกเซลใหม่ทั้งภาพ ทำให้แยกไม่ออกว่าผลที่วัดเกิดจากการซ่อนข้อมูลหรือจากการย่อขยาย

2.3 ผลก่อน/หลังการแก้ไขข้อมูล

ตารางที่ 2.6 ผลก่อน/หลังการแก้ไขข้อมูล

	ก่อนแก้ไข	หลังแก้ไข
ชนิดข้อมูลต่อภาพ	ภาพสี RGB (3 ช่อง)	เมทริกซ์จำนวนจริง 1 ช่อง
ขนาด (ภาพแนวนอน)	481 × 321 × 3 = 463,203 ค่า	320 × 480 = 153,600 ค่า
พิกเซลที่ถูกตัดทิ้งจากการตัดขอบ	—	154,401 – 153,600 = 801 พิกเซล (0.52%)
ช่วงค่า	0–255 ต่อช่องสี	0–255 (ความสว่าง)
จำนวนบล็อก 8×8 ต่อภาพ	แบ่งไม่ลงตัว	40 × 60 = 2,400 บล็อก
จำนวนภาพที่ใช้ได้	500	500 (ไม่มีภาพถูกตัดทิ้ง)



รูปที่ 2.2 ภาพ 249061.jpg ก่อนแก้ไข (ภาพสี 481×321) และหลังแก้ไข (เมทริกซ์ภาพเทา 320×480)

2.4 การแบ่งชุดข้อมูล

เพื่อไม่ให้ "เลือกค่าพารามิเตอร์จากข้อมูลชุดเดียวกับที่ใช้รายงานผล" (ซึ่งเหมือนดูเฉลยก่อนตอบ ทำให้ผลดีเกินจริง) จึงแบ่งภาพเป็นสองชุดที่ไม่มีภาพซ้ำกันเลย

ตารางที่ 2.7 การแบ่งชุดข้อมูล (รายชื่อใน `results/image_sets.csv`) โครงงานนี้ไม่ได้ฝึกโมเดล ชื่อกลุ่มของ BSDS500 จึงใช้เป็นแค่การแบ่งที่ตายตัว

ชุด	ภาพ	จำนวน	ใช้ทำอะไร
pilot	กลุ่ม val ของ BSDS500	100	ทดลองนำร่องเพื่อเลือกพารามิเตอร์ b, Δ, k, α เท่านั้น
eval	กลุ่ม train + test	400	ตัวเลขผลลัพธ์ทุกตัวในบทที่ 6 (ไม่เคยใช้เลือกค่า)

บทที่ 3
การนำข้อมูลเข้าสู่โครงงาน: เวกเตอร์และเมทริกซ์

3.1 ภาพเป็นเมทริกซ์

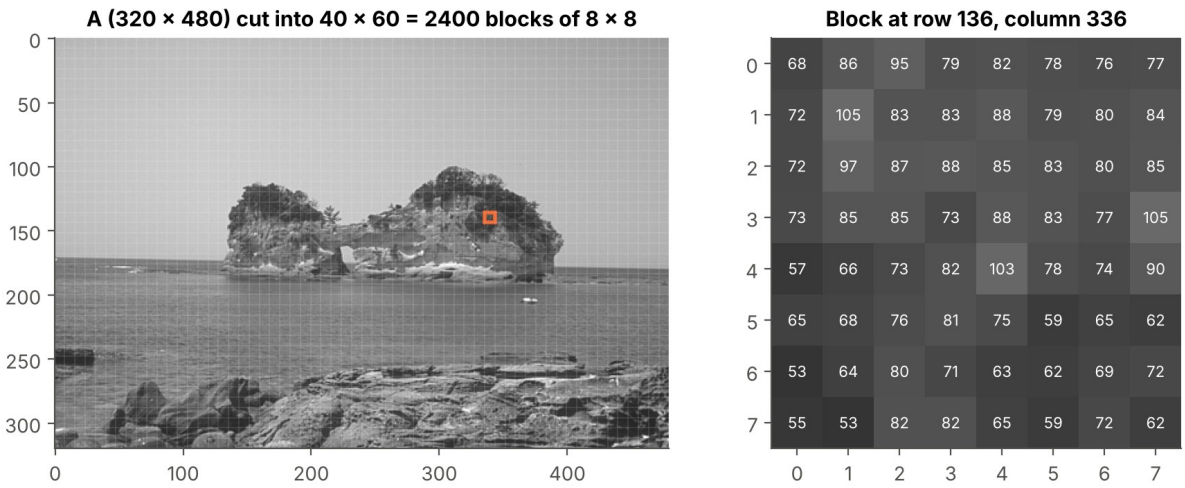
ภาพเทาขนาด 320×480 คือเมทริกซ์ $A \in R^{320 \times 480}$ โดยสมาชิก a_{ij} คือความสว่างของพิกเซลแถว i คอลัมน์ j (0 = ดำ, 255 = ขาว) เช่น มุมบนซ้ายของภาพ 249061.jpg (ท้องฟ้า) คือ

$$A_{[0:4,0:6)} = \begin{bmatrix} 182 & 182 & 183 & 183 & 182 & 181 \\ 182 & 183 & 184 & 184 & 183 & 182 \\ 182 & 183 & 184 & 185 & 184 & 183 \\ 183 & 183 & 184 & 184 & 184 & 184 \end{bmatrix}$$

3.2 แบ่งเป็นบล็อกสำหรับการซ่อนข้อความ

งานซ่อนข้อความแบ่ง A เป็นบล็อกขนาด $b \times b$ ที่ไม่ทับกัน เรียงจากซ้ายไปขวา บนลงล่าง (raster order) แต่ละบล็อกคือเมทริกซ์ $B_i \in R^{b \times b}$ และซ่อนได้ 1 บิต ความจุจึงเท่ากับจำนวนบล็อก

$$\text{capacity} = \frac{320}{b} \cdot \frac{480}{b} : b=4 \Rightarrow 9{,}600,\ b=8 \Rightarrow 2{,}400,\ b=16 \Rightarrow 600 \text{ บิต}$$



รูปที่ 3.1 เมทริกซ์ภาพถูกแบ่งเป็นตารางบล็อก 8×8 (เส้นบาง) และตัวอย่างบล็อก B ที่แถว 136 คอลัมน์ 336 (กรอบส้ม) แสดงค่าพิกเซลจริงทั้ง 64 ค่า

บล็อกตัวอย่างนี้ใช้ตลอดรายงาน เมื่อทำ SVD ได้ค่าเอกฐานเป็น**เวกเตอร์** $\sigma=(616.92, 37.67, 35.01, 22.46, 10.88, 9.88, 4.53, 2.07)$ ค่า σ_1 ใหญ่กว่าค่าอื่นมาก (เก็บพลังงาน $616.92^2/\sum\sigma_i^2=99.1\%$ ของบล็อก)

3.3 ข้อมูลอื่นที่อยู่ในรูปเวกเตอร์/เมทริกซ์

ตารางที่ 3.1 รูปแบบข้อมูลก่อนเข้าสู่การประมวลผล

ข้อมูล	รูปแบบ	ขนาด
ภาพเทา	เมทริกซ์ A	320×480 (หรือ 480×320)
บล็อกภาพ	เมทริกซ์ B_i	$b \times b$

ข้อมูล	รูปแบบ	ขนาด
ค่าเอกฐานของบล็อก/ภาพ	เวกเตอร์ σ	b ค่า / 320 ค่า (เก็บของ 400 ภาพไว้ใน `sigma_eval.csv` เป็นเมทริกซ์ 400×320)
ข้อความลับ	เวกเตอร์บิต $m \in \{0,1\}^n$	32 บิตหัวข้อความ + 8 บิตต่อไบต์
รหัส Hamming	เวกเตอร์ 4 บิต $d \to 7$ บิต c, เมทริกซ์ G (4×7), H (3×7)	คิดเลขแบบ mod 2
ลายน้ำ	เมทริกซ์ขาวดำ $W \in \{0,1\}^{320 \times 320}$	ขนาดเท่า Σ ของภาพ

3.4 ค่าที่ใช้วัดผล (นิยามด้วยเวกเตอร์/เมทริกซ์)

- **MSE และ PSNR** (คุณภาพภาพ): $MSE = \frac{1}{mn} \| A' - A \|_F^2$, $PSNR = 10 \log_{10} \frac{255^2}{MSE}$ dB ยิ่งสูงยิ่งเหมือน ค่า ≥ 40 dB ถือว่าตามองไม่ออก
- **BER** (อัตราบิตผิด) = จำนวนบิตที่อ่านผิด ÷ จำนวนบิตทั้งหมด
- **NCC** (ความคล้ายของลายน้ำ) = cosine ของมุมระหว่าง W กับ $W^{¿}$ เมื่อมองเป็นเวกเตอร์ยาว 320^2: $NCC = \langle W, W^{¿} \rangle / (\| W \| \| W^{¿} \|)$

- AUC (ความสามารถของตัวตรวจในการแยกสองกลุ่ม) = ความน่าจะเป็นที่ภาพจากกลุ่ม "ถูกแก้" ได้คะแนนสูงกว่าภาพจากกลุ่ม "ปกติ" (0.5 = เดาสุ่ม, 1 = แยกได้สมบูรณ์)

บทที่ 4
ทฤษฎีเวกเตอร์และเมทริกซ์ที่ใช้ พร้อมตัวอย่างการคำนวณ

บทนี้แบ่งเป็นสองส่วนตามที่โจทย์กำหนด: **4.1 ทฤษฎีเวกเตอร์** และ **4.2 ทฤษฎีเมทริกซ์** แต่ละหัวข้อบอกทฤษฎี ตัวอย่างคำนวณละเอียด และจุดที่ใช้ในโครงงาน จากนั้น 4.3–4.5 แสดงการประยุกต์รวมกันในงานจริงทีละขั้น โค้ดของแต่ละทฤษฎีวางไว้ต่อจากตัวอย่างคำนวณมือ พร้อมภาพผลรันจริง (สคริปต์อยู่ในโฟลเดอร์ `examples/` ของ repository) และตัวอย่างทุกข้อตรวจซ้ำด้วยเทสต์ในไฟล์ `tests/test_hand_examples.py` และ `tests/test_ecc.py`

4.1 ทฤษฎีเวกเตอร์

4.1.1 ความยาว (norm) และเวกเตอร์หนึ่งหน่วย

ความยาวของเวกเตอร์ $x=(x_1,\ldots,x_n)$ คือ $\|x\|=\sqrt{x_1^2+\cdots+x_n^2}$ เวกเตอร์ที่ยาว 1 เรียกว่าเวกเตอร์หนึ่งหน่วย ได้จากการหารด้วยความยาว $\hat{x}=x/\|x\|$

ตัวอย่าง $1=(1,1,1)$ มีความยาว $\sqrt{1+1+1}=\sqrt{3}\approx 1.732$ เวกเตอร์หนึ่งหน่วยคือ $1/\sqrt{3}=(0.577,0.577,0.577)$ ตรวจ: $\sqrt{3\times 0.577^2}=\sqrt{3\times 0.3333}=1$ ✓

ใช้ที่: เวกเตอร์เอกฐาน u_i, v_i ทุกตัวยาว 1 (ข้อ 4.2.4) และในบล็อกเรียบ $u_1 \approx 1/\sqrt{8}$ (ข้อ 6.6)

4.1.2 ผลคูณภายใน (dot product) และความตั้งฉาก

$\langle x, y \rangle = x^T y = \sum_i x_i y_i$ ถ้าผลคูณภายในเป็น 0 เวกเตอร์สองตัวตั้งฉากกัน ชุดเวกเตอร์ที่ยาว 1 และตั้งฉากกันทุกคู่เรียกว่า **orthonormal**

ตัวอย่าง $v_1 = 1/\sqrt{2}(1,1)$, $v_2 = 1/\sqrt{2}(1,-1)$:
$\langle v_1, v_2 \rangle = 1/2(1 \cdot 1 + 1 \cdot (-1)) = 0$ (ตั้งฉาก) และ $\|v_1\|^2 = 1/2(1+1) = 1$ (ยาว 1) จึงเป็น orthonormal

ใช้ที่: คอลัมน์ของ U และ V ใน SVD เป็น orthonormal ซึ่งทำให้การเปลี่ยน σ ควบคุมขนาดการเปลี่ยนภาพได้ (ข้อ 4.2.5)

4.1.3 มุมระหว่างเวกเตอร์ (cosine similarity)

$$\cos\theta = \frac{\langle x, y \rangle}{\|x\|\|y\|} \in [-1, 1)$$

ค่า 1 = ชี้ทางเดียวกัน, 0 = ตั้งฉาก (ไม่เกี่ยวกัน), −1 = ชี้สวนทาง

ตัวอย่าง (NCC ของลายน้ำ) ลายน้ำ W และค่าที่ถอดได้ $W^{\wr}$ ขนาด 3×3 มองแต่ละตัวเป็นเวกเตอร์ยาว 9

$$W = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}, W^{\wr} = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix}$$

ผลคูณภายในนับเฉพาะช่องที่เป็น 1 ทั้งคู่ (มุมขวาบนและมุมซ้ายล่าง)

$$\langle W, W^{\wr} \rangle = 1 \cdot 1 + 1 \cdot 1 = 2, \|W\| = \sqrt{5}, \|W^{\wr}\| = \sqrt{2}$$

$$NCC = \frac{\langle W, W^{¿} \rangle}{\|W\| \, \|W^{¿}\|} = \frac{2}{\sqrt{5}\sqrt{2}} = \frac{2}{3.162} = 0.632$$

ใช้ที่: NCC ของลายน้ำ (ข้อ 4.4) และ shift score ที่เราเสนอ = cosine ระหว่างเวกเตอร์ $\sigma^{¿} - \sigma$ กับ $\sigma_w - \sigma$ (ข้อ 4.4.3)

4.1.4 เวกเตอร์บนตัวเลข 0/1 (การคิดเลขแบบ mod 2)

บิตมีค่าแค่ 0 กับ 1 ถ้าบวกแล้วหารด้วย 2 เอาเศษ ($1+1=0$) การบวกจะเหมือน XOR และเวกเตอร์บิตก็บวก คูณสเกลาร์ และคูณเมทริกซ์ได้เหมือนเวกเตอร์จำนวนจริง

ตัวอย่าง $(1,0,1,1)+(1,1,0,1)=(0,1,1,0)$ (mod 2) และผลคูณภายใน $(1,0,1,1)\cdot(1,1,0,1)=1+0+0+1=2\equiv 0$

ใช้ที่: รหัสแก้บิตผิด Hamming(7,4) (ข้อ 4.5)

4.1.5 ตรวจตัวอย่างในหัวข้อ 4.1 ด้วยโค้ด

สคริปต์ด้านล่างคำนวณตัวอย่างทั้งสี่ข้อของหัวข้อ 4.1 ด้วย NumPy และฟังก์ชัน `ncc` ของโครงงาน (ไฟล์ `metrics.py`) ผลที่ได้ตรงกับที่คำนวณมือ: ความยาว $\sqrt{3}=1.7321$, ผลคูณภายในของ v_1, v_2 เป็น 0, NCC = 0.632 และผลบวกแบบ mod 2

โค้ด 4.1 `examples/ex1_vectors.py`

```
import numpy as np

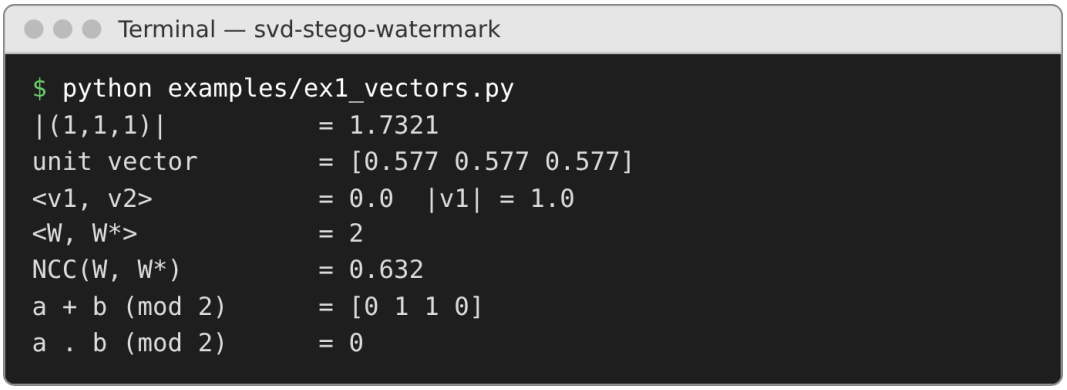
from svdmark.metrics import ncc

ones = np.array([1.0, 1.0, 1.0])
print("|(1,1,1)|          =", round(np.linalg.norm(ones), 4))
print("unit vector        =", np.round(ones /
np.linalg.norm(ones), 3))

v1 = np.array([1, 1]) / np.sqrt(2)
v2 = np.array([1, -1]) / np.sqrt(2)
print(
    "<v1, v2>           =",
    abs(round(v1 @ v2, 10)),
    " |v1| =",
    round(np.linalg.norm(v1), 4),
)

W = np.array([[1, 0, 1], [0, 1, 0], [1, 0, 1]])
W_star = np.array([[0, 0, 1], [0, 0, 0], [1, 0, 0]])
print("<W, W*>            =", int(W.ravel() @ W_star.ravel()))
print("NCC(W, W*)         =", round(ncc(W, W_star), 3))

a, b = np.array([1, 0, 1, 1]), np.array([1, 1, 0, 1])
print("a + b (mod 2)      =", (a + b) % 2)
print("a . b (mod 2)      =", (a @ b) % 2)
```



รูปที่ 4.1 ผลรันโค้ด 4.1 ตรงกับตัวอย่างคำนวณมือในหัวข้อ 4.1.1–4.1.4

4.2 ทฤษฎีเมทริกซ์

4.2.1 การคูณเมทริกซ์ และ transpose

$(AB)_{ij}=\sum_k a_{ik}b_{kj}$ และ $(A^T)_{ij}=a_{ji}$ มีสมบัติ $(AB)^T=B^TA^T$ ผลคูณ "คอลัมน์ × แถว" uv^T (outer product) ให้เมทริกซ์ที่มี rank 1

ตัวอย่าง หา A^TA ของ A ด้านล่าง: ช่อง (i,j) ของผลคูณ = แถว i ของ A^T คูณกับคอลัมน์ j ของ A

$$A=\begin{bmatrix}3&0\\4&5\end{bmatrix},A^TA=\begin{bmatrix}3&4\\0&5\end{bmatrix}\begin{bmatrix}3&0\\4&5\end{bmatrix}=\begin{bmatrix}3\cdot3+4\cdot4&3\cdot0+4\cdot5\\0\cdot3+5\cdot4&0\cdot0+5\cdot5\end{bmatrix}=\begin{bmatrix}25&20\\20&25\end{bmatrix}$$

outer product ของเวกเตอร์ $(1,1,1)$ กับตัวเอง ได้เมทริกซ์ rank 1 ที่ทุกช่องเท่ากัน

$$\begin{bmatrix}1\\1\\1\end{bmatrix}\begin{bmatrix}1&1&1\end{bmatrix}=\begin{bmatrix}1&1&1\\1&1&1\\1&1&1\end{bmatrix}$$

4.2.2 เมทริกซ์เชิงตั้งฉาก (orthogonal matrix)

เมทริกซ์จัตุรัส Q ที่คอลัมน์เป็น orthonormal มีสมบัติ $Q^TQ=I$ (จึง $Q^{-1}=Q^T$) และ **ไม่เปลี่ยนความยาว**: $\|Qx\|=\|x\|$ เพราะ $\|Qx\|^2=x^TQ^TQx=x^Tx$

4.2.3 ค่าลักษณะเฉพาะและเวกเตอร์ลักษณะเฉพาะ

λ และ $x\neq0$ ที่ทำให้ $Mx=\lambda x$ หาได้จาก $det(M-\lambda I)=0$ เมทริกซ์สมมาตรอย่าง A^TA มีค่าลักษณะเฉพาะเป็นจำนวนจริงไม่ติดลบ และเวกเตอร์ลักษณะเฉพาะตั้งฉากกัน

4.2.4 การแยกค่าเอกฐาน (SVD)

ทุกเมทริกซ์ $A \in R^{m \times n}$ เขียนได้เป็น $A = U \Sigma V^T$ โดย U และ V เป็นเมทริกซ์เชิงตั้งฉาก และ Σ เป็นเมทริกซ์ทแยงของ $\sigma_1 \geq \sigma_2 \geq \ldots \geq 0$ หาได้ดังนี้: (1) v_i คือเวกเตอร์ลักษณะเฉพาะ (ยาว 1) ของ $A^T A$ (2) $\sigma_i = \sqrt{\lambda_i}$ (3) $u_i = A v_i / \sigma_i$

ตัวอย่างที่ 1: SVD ของเมทริกซ์ 2×2 ด้วยมือ

$$A = \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix}$$

ขั้นที่ 1 คำนวณ $A^T A$ (ทำไว้แล้วในข้อ 4.2.1)

$$A^T A = \begin{bmatrix} 25 & 20 \\ 20 & 25 \end{bmatrix}$$

ขั้นที่ 2 หาค่าลักษณะเฉพาะจาก $det(A^T A - \lambda I) = 0$

$$det \begin{bmatrix} 25-\lambda & 20 \\ 20 & 25-\lambda \end{bmatrix} = (25-\lambda)^2 - 400 = 0 \Rightarrow 25 - \lambda = \pm 20 \Rightarrow \lambda_1 = 45, \lambda_2 = 5$$

ขั้นที่ 3 ค่าเอกฐานคือรากที่สองของค่าลักษณะเฉพาะ $\sigma_i = \sqrt{\lambda_i}$

$$\sigma_1 = \sqrt{45} = 3\sqrt{5} = 6.7082, \sigma_2 = \sqrt{5} = 2.2361$$

ขั้นที่ 4 เวกเตอร์ลักษณะเฉพาะ: แก้ $(A^T A - \lambda I)x = 0$ แล้วหารด้วยความยาวให้ยาว 1

$$\lambda_1 = 45: \quad \begin{bmatrix} -20 & 20 \\ 20 & -20 \end{bmatrix} x = 0 \Rightarrow x = (1,1) \Rightarrow v_1 = 1/\sqrt{2}(1,1)$$

$$\lambda_2 = 5: \quad \begin{bmatrix} 20 & 20 \\ 20 & 20 \end{bmatrix} x = 0 \Rightarrow x = (1,-1) \Rightarrow v_2 = 1/\sqrt{2}(1,-1)$$

ขั้นที่ 5 เวกเตอร์ทางซ้าย $u_i = A v_i / \sigma_i$ (เช่น $A(1,1) = (3,9)$)

$$u_1 \quad ¿ \frac{1}{3\sqrt{5}} \cdot \frac{1}{\sqrt{2}} \begin{bmatrix} 3 \\ 9 \end{bmatrix} = \frac{1}{\sqrt{10}} \begin{bmatrix} 1 \\ 3 \end{bmatrix} = \begin{bmatrix} 0.3162 \\ 0.9487 \end{bmatrix}$$

$$u_2 \quad ¿ \frac{1}{\sqrt{5}} \cdot \frac{1}{\sqrt{2}} \begin{bmatrix} 3 \\ -1 \end{bmatrix} = \frac{1}{\sqrt{10}} \begin{bmatrix} 3 \\ -1 \end{bmatrix} = \begin{bmatrix} 0.9487 \\ -0.3162 \end{bmatrix}$$

ขั้นที่ 6 ตรวจ: ผลรวมของสองชั้น rank 1 ต้องได้ A คืน ($3\sqrt{5}/\sqrt{20}=1.5$ และ $\sqrt{5}/\sqrt{20}=0.5$)

$$\sigma_1 u_1 v_1^T + \sigma_2 u_2 v_2^T \quad ¿ \quad \frac{3\sqrt{5}}{\sqrt{20}}\begin{bmatrix}1 & 1\\ 3 & 3\end{bmatrix} + \frac{\sqrt{5}}{\sqrt{20}}\begin{bmatrix}3 & -3\\ -1 & 1\end{bmatrix}$$

¿ ¿

NumPy ให้ $\sigma=(6.7082, 2.2361)$ ตรงกัน (เครื่องหมายของ u, v อาจกลับทั้งคู่ ซึ่งไม่เปลี่ยนผลคูณ) ตรวจใน `test_ex1_*` และโค้ดคำนวณมือ `svd_2x2_by_hand`

ในโครงงาน SVD คำนวณด้วย NumPy ผ่านสองฟังก์ชันนี้ ซึ่งเป็นสูตร $A=U\Sigma V^T$ ตรงๆ

โค้ด 4.2 SVD และการสร้างเมทริกซ์กลับ (`svd_utils.py`)

```
def svd(A):
    """Thin SVD, A = U @ diag(s) @ Vt, with s sorted high ->
low."""
    return np.linalg.svd(np.asarray(A, dtype=float),
full_matrices=False)

def reconstruct(U, s, Vt):
    """Rebuild a matrix from its SVD factors: U diag(s)
V^T."""
    return (U * s) @ Vt
```

สคริปต์ต่อไปนี้ทำตัวอย่างที่ 1 สองวิธี วิธีแรกตามขั้นตอนมือ (หาค่าลักษณะเฉพาะของ A^TA แล้วถอดราก) ด้วยฟังก์ชัน `svd_2x2_by_hand` วิธีที่สองด้วย NumPy แล้วสร้าง A กลับจากสองชั้น rank 1 และตรวจพลังงานในหัวข้อ 4.2.5 ไปพร้อมกัน

โค้ด 4.3 `examples/ex2_svd_2x2.py`

```
import numpy as np

from svdmark.svd_utils import reconstruct, svd,
svd_2x2_by_hand

A = np.array([[3.0, 0.0], [4.0, 5.0]])
print("A^T A =\n", A.T @ A)
print("eigenvalues of A^T A =", np.linalg.eigvalsh(A.T @ A)
[::-1])

U, s, Vt = svd_2x2_by_hand(A)
print("by hand: sigma =", np.round(s, 4))
print("         u1 =", np.round(U[:, 0], 4), " v1 =",
np.round(Vt[0], 4))
print("numpy:   sigma =", np.round(svd(A)[1], 4))

print(
    "sigma1 u1 v1^T + sigma2 u2 v2^T =\n",
    np.round(reconstruct(U, s, Vt), 10),
)
print(
    "|A|_F^2 =",
    np.sum(A**2),
    "  sigma1^2 + sigma2^2 =",
    round(np.sum(s**2), 10),
)
```

```
Terminal — svd-stego-watermark
$ python examples/ex2_svd_2x2.py
A^T A =
 [[25. 20.]
 [20. 25.]]
eigenvalues of A^T A = [45.  5.]
by hand: sigma = [6.7082 2.2361]
         u1 = [0.3162 0.9487]  v1 = [0.7071 0.7071]
numpy:   sigma = [6.7082 2.2361]
sigma1 u1 v1^T + sigma2 u2 v2^T =
 [[3. 0.]
 [4. 5.]]
|A|_F^2 = 50.0   sigma1^2 + sigma2^2 = 50.0
```

รูปที่ 4.2 ผลรันโค้ด 4.3: ได้ $\lambda = 45, 5$, $\sigma = (6.7082, 2.2361)$, $u_1 = (0.3162, 0.9487)$ และสร้าง A กลับได้ครบ ตรงกับขั้นที่ 1–6 ของตัวอย่างที่ 1

4.2.5 Frobenius norm, พลังงาน และการไม่เปลี่ยนแปลงภายใต้เมทริกซ์เชิงตั้งฉาก

$\|A\|_F^2 = \sum_{ij} a_{ij}^2 = \sum_i \sigma_i^2$ (เรียกว่า "พลังงาน") เพราะ U, V ไม่เปลี่ยนความยาว ผลที่สำคัญที่สุดต่อโครงงานคือ ถ้าเปลี่ยนแค่ Σ เป็น Σ' โดยคง U, V ไว้

$$\|U\Sigma' V^T - U\Sigma V^T\|_F = \|U(\Sigma' - \Sigma)V^T\|_F = \|\Sigma' - \Sigma\|_F$$

ตัวอย่าง $A = \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix}$: $\|A\|_F^2 = 9+0+16+25 = 50$ และ $\sigma_1^2 + \sigma_2^2 = 45+5 = 50$ ✓

ผลที่ตามมา: ถ้าเปลี่ยน σ_1 ของบล็อก $b \times b$ ไป d ผลรวมกำลังสองของการเปลี่ยนพิกเซลทั้ง b^2 ช่อง ¿d^2 ดังนั้นพิกเซลเปลี่ยนเฉลี่ย (RMS) ¿d/b เช่น $b=8, d=16 \rightarrow$ พิกเซลเปลี่ยนเฉลี่ย 2 ระดับสี

ตรวจกับภาพจริง: $\|A_w - A\|_F$ เทียบกับ $\|\sigma_w - \sigma\|$ ของ 400 ภาพต่างกันมากสุด 6.9×10^{-14} (เท่ากับความคลาดเคลื่อนของการคำนวณทศนิยม)

4.2.6 ค่า σ_1 กับความสว่างเฉลี่ยของบล็อก

จากนิยาม $\sigma_1 = max_{\|x\|=\|y\|=1} x^T B y$ ถ้าเลือก $x = y = 1/\sqrt{b}$ จะได้ $x^T B y = 1/b \sum_{ij} b_{ij} = b \cdot \acute{m}$ ($\acute{m}$ = ความสว่างเฉลี่ย) ดังนั้น $\sigma_1 \geq b\acute{m}$ เสมอ และเท่ากันพอดีถ้าบล็อกเรียบ (ทุกพิกเซลเท่ากับ m) เพราะเมทริกซ์นั้นเป็นชั้น rank 1 ชั้นเดียว

$$B = m\,1\,1^T = (mb)\frac{1}{\sqrt{b}}\frac{1^T}{\sqrt{b}} \Rightarrow \sigma_1 = mb$$

ตัวอย่าง (บล็อกจริงในรูปที่ 3.1) ความสว่างเฉลี่ย $\acute{m}=76.55$ ได้ $b\acute{m}=8\times 76.55=612.4$ ขณะที่ $\sigma_1=616.9$ ✓ (มากกว่าเล็กน้อย) และ $u_1=(0.368, 0.387, 0.389, 0.384, 0.359, 0.316, 0.307, 0.305)$ (กลับเครื่องหมายแล้ว) ใกล้ $1/\sqrt{8}=0.354$ ทุกตัว

ผลที่ตามมา: การเปลี่ยน σ_1 เกือบเท่ากับการเลื่อนความสว่างทั้งบล็อก ข้อนี้อธิบายว่าทำไม σ_1 ทนการรบกวนได้ดี และทำไมตัวตรวจจับเห็นรอยที่ขอบบล็อก (ข้อ 6.6)

4.2.7 ทฤษฎีบท Weyl (ความไวของค่าเอกฐาน)

$$\left|\sigma_i(A+E)-\sigma_i(A)\right|\le \|E\|_2=\sigma_1(E) \text{ ทุก } i$$

ความหมาย: ถ้าภาพถูกรบกวนด้วย E ค่าเอกฐานทุกตัวขยับไม่เกิน "ขนาดใหญ่สุด" ของการรบกวน ใช้ได้สองแบบ: (1) การปัดเศษพิกเซลทำให้แต่ละพิกเซลคลาดไม่เกิน 0.5 จึง $\|E\|_2\le\|E\|_F\le 0.5b$ ถ้าช่วง Δ ใหญ่กว่า b การปัดเศษจะไม่ทำให้บิตพลิก (2) ทำนายผลของสัญญาณรบกวน: ถ้าแต่ละพิกเซลถูกบวกด้วยสัญญาณรบกวนส่วนเบี่ยงเบน s ค่า σ_1 จะขยับแบบสุ่มด้วยส่วนเบี่ยงเบนประมาณ s (เพราะ $\delta\sigma_1\approx u_1^T E v_1$ เป็นผลรวมถ่วงน้ำหนักที่น้ำหนักรวมกำลังสองได้ 1) บิตจะพลิกเมื่อขยับเกินครึ่งช่วง

$$BER\approx 2Q\left(\frac{\Delta}{2s}\right), Q(z)=P(Z>z), Z\sim N(0,1)$$

ตัวอย่าง $\Delta=16,\ s=4$: $\Delta/2s=2$, $Q(2)=0.02275$ ได้ BER ≈ 4.55% วัดจริงได้ 4.67% · ตรวจ Weyl บนบล็อก 8×8 จริง: อัตราส่วน $max_i|\delta\sigma_i)/\|E\|_2$ สูงสุด 0.997 ≤ 1 ✓

4.3 การประยุกต์ที่ 1: ซ่อนข้อความใน σ_1 ของบล็อก (QIM)

4.3.1 กฎการซ่อน 1 บิต (interval-parity QIM)

แบ่งแกนของ σ เป็นช่วงกว้าง Δ ช่วงที่ $q=\lfloor\sigma/\Delta\rfloor$ ช่วงเลขคู่แทนบิต 0 ช่วงเลขคี่แทนบิต 1 การ**อ่าน**บิตคือ $\text{bit}=q \bmod 2$ การ**ซ่อน**บิตคือย้าย σ ไปที่ **กึ่งกลาง** $(q'+1/2)\Delta$ ของช่วงที่ใกล้ที่สุดซึ่งพาริตี้ตรงกับบิต การไว้ที่กึ่งกลางทำให้ σ ขยับได้ถึง $\pm\Delta/2$ โดยยังอ่านบิตเดิม

ตัวอย่างที่ 3: ซ่อนบิต 1 ในบล็อก 2×2 ด้วย $\Delta=6$

$$B=\begin{bmatrix}52 & 55\\ 61 & 59\end{bmatrix},\ B^TB=\begin{bmatrix}52^2+61^2 & 52\cdot55+61\cdot59\\ 55\cdot52+59\cdot61 & 55^2+59^2\end{bmatrix}=\begin{bmatrix}6425 & 6459\\ 6459 & 6506\end{bmatrix}$$

ค่าเอกฐาน: ค่าลักษณะเฉพาะของเมทริกซ์สมมาตร 2×2 หาได้จากสูตร แล้วถอดราก

$$\lambda=\frac{6425+6506}{2}\pm\sqrt{(\frac{6425-6506}{2})^2+6459^2}=6465.5\pm6459.13$$

$$\lambda_1=12924.63,\ \lambda_2=6.37$$

$$\sigma_1=\sqrt{12924.63}=113.687,\ \sigma_2=\sqrt{6.37}=2.524$$

อ่านบิตเดิม: $q=\lfloor 113.687/6\rfloor=\lfloor 18.95\rfloor=18$ (คู่) → บิต 0 แต่ต้องการบิต 1 จึงย้ายไปช่วงคี่ที่ใกล้ที่สุด คือช่วง 19 ซึ่งมีกึ่งกลาง $(19+1/2)\times6=117$ (ช่วง 17 มีกึ่งกลาง 105 ไกลกว่า)

สร้างบล็อกใหม่: ใส่ $\sigma_1 = 117$ แทน 113.687 โดยคง u_i, v_i เดิม แล้วปัดเป็นจำนวนเต็ม

$$B' = 117\,u_1 v_1^T + 2.524\,u_2 v_2^T = \begin{bmatrix} 53.55 & 56.56 \\ 62.74 & 60.75 \end{bmatrix} \xrightarrow{\text{round}} \begin{bmatrix} 54 & 57 \\ 63 & 61 \end{bmatrix}$$

ผู้รับอ่าน (closed-loop): SVD ของบล็อกที่ปัดแล้วได้ $\sigma_1 = 117.68$

$\rightarrow \lfloor 117.68/6 \rfloor = 19$ (คี่) → บิต 1 ✓

คุณภาพ: ทุกพิกเซลเปลี่ยน $+2$ → $MSE = 4$ →

$PSNR = 10\log_{10}(255^2/4) = 10\log_{10} 16256.25 = 42.11$ dB

โค้ด 4.4 กฎซ่อนและอ่าน 1 บิต (`stego.py`)

```
def extract_bit(sigma, delta):
    """Bit carried by sigma: parity of the interval
floor(sigma/delta)."""
    return int(np.floor(sigma / delta)) % 2

def embed_value(sigma, bit, delta):
    """Centre of the nearest interval whose parity equals
`bit`."""
    q = int(np.floor(sigma / delta))
    if q % 2 != bit:
        up, down = q + 1, q - 1
        if down < 0 or (abs((up + 0.5) * delta - sigma)
                        <= abs((down + 0.5) * delta - sigma)):
            q = up
        else:
            q = down
    return (q + 0.5) * delta
```

โค้ด 4.5 **examples/ex3_qim_block.py** (ตัวอย่างที่ 3)

```
import numpy as np

from svdmark.metrics import psnr
from svdmark.stego import embed_value, extract_bit
from svdmark.svd_utils import reconstruct, svd

B = np.array([[52.0, 55.0], [61.0, 59.0]])
U, s, Vt = svd(B)
print("sigma            =", np.round(s, 3))
print(
    "bit now          =",
    extract_bit(s[0], 6),
    f"(interval {int(s[0] // 6)})",
)

s_new = s.copy()
s_new[0] = embed_value(s[0], 1, 6)  # centre of the nearest
odd interval
print("new sigma1       =", s_new[0])
B_new = reconstruct(U, s_new, Vt)
print("B' (before round)=\n", np.round(B_new, 2))
B_int = np.rint(B_new)
print("B' (rounded)     =\n", B_int)

s_read = svd(B_int)[1][0]  # what the receiver sees
print(
    "receiver sigma1  =",
    round(s_read, 2),
    " -> bit",
    extract_bit(s_read, 6),
)
print("PSNR             =", round(psnr(B, B_int), 2), "dB")
```

```
Terminal — svd-stego-watermark
$ python examples/ex3_qim_block.py
sigma            = [113.687   2.524]
bit now          = 0 (interval 18)
new sigma1       = 117.0
B' (before round)=
 [[53.55 56.56]
 [62.74 60.75]]
B' (rounded)     =
 [[54. 57.]
 [63. 61.]]
receiver sigma1  = 117.68  -> bit 1
PSNR             = 42.11 dB
```

รูปที่ 4.3 ผลรันโค้ด 4.5: σ_1 ย้ายจาก 113.687 ไป 117 บล็อกที่ปัดแล้วอ่านได้ $\sigma_1 = 117.68$ เป็นบิต 1 และ PSNR 42.11 dB ตรงกับตัวอย่างที่ 3

> **ทำไมไม่ใช้สูตรปัดเศษ** ถ้าอ่านบิตด้วย $round(\sigma/\Delta) \bmod 2$ (แบบที่พบในงานเก่าบางชิ้น) จะไม่ตรงกับกฎการซ่อนแบบช่วง เช่น $\Delta = 10, \sigma = 37$: ช่วงที่ 3 (คี่) → บิต 1 แต่ $round(3.7) = 4 \rightarrow$ บิต 0 (ตรวจใน `test_qim_interval_parity_vs_rounding`)

4.3.2 ปัญหาการสลับลำดับ และเหตุผลที่ใช้ σ_1

SVD เรียง σ จากมากไปน้อยเสมอ ถ้าตั้ง σ_k ให้ต่ำกว่า σ_{k+1} ตำแหน่งจะสลับกัน และผู้รับอ่านค่าผิดตัว ตัวอย่างจริงจากบล็อกในรูปที่ 3.1 (ซ่อนบิต 1 ใน σ_2, $\Delta = 16$): ตั้ง $\sigma_2 = 24$ (ต่ำกว่า $\sigma_3 = 35.0$) แล้วทำ SVD ใหม่ได้ลำดับ $(616.9, 34.9, 24.5, \ldots)$ คือ σ_2 ใหม่กลายเป็น 34.9 → บิต 0 ผิด จึงต้องลองช่วงถัดไป (ตั้งเป็น 56 แล้วอ่านได้ 55.9 → บิต 1 ✓) σ_1 ไม่มีปัญหานี้ เพราะใหญ่กว่าตัวอื่นมาก (ในทุกการทดลอง σ_1 ไม่เคยล้มเหลว ส่วน σ_3, σ_4 ล้มเหลว 3–50% ของบล็อก)

4.3.3 ตรวจซ้ำแบบ closed-loop พร้อมระยะเผื่อ

สำหรับทุกบล็อก ระบบลองตำแหน่งกึ่งกลางที่ถูกพาริตี้ เรียงจากใกล้ไปไกล (สูงสุด ±3 ช่วง) แต่ละตัวเลือก: สร้างบล็อกใหม่ → ปัดเป็นจำนวนเต็มและตัดให้อยู่ใน 0–255 → ทำ SVD → อ่านบิตแบบเดียวกับผู้รับ ยอมรับเมื่อ (1) อ่านได้บิตถูก **และ** (2) σ ที่อ่านได้อยู่ห่างขอบช่วงอย่างน้อย $\Delta/4$ ถ้าไม่มีตัวเลือกใดผ่านเงื่อนไข (2) จะใช้ตัวที่อ่านถูกและห่างขอบมากที่สุด ถ้าไม่มีตัวที่อ่านถูกเลย บล็อกนั้นนับว่า "ล้มเหลว" และไม่ถูกแก้ ระยะเผื่อเพิ่มเข้ามาหลังพบว่าบล็อกที่พิกเซลชน 255 ถูกปัดจนไปตกชิดขอบช่วง (บทที่ 7)

โค้ด 4.6 ส่วนหลักของการซ่อนพร้อมตรวจซ้ำ (ฟังก์ชัน `embed_bits` ใน `stego.py`)

```
U, s, Vt = np.linalg.svd(sel)                          # SVD ของ
ทุกบล็อกพร้อมกัน
sk = s[:, k]                                           # sigma ที่
ใช้ซ่อน (k = 0: sigma_1)
u = _dither(n, delta, key)                             # ระยะ
เลื่อนเส้นแบ่งจากรหัสลับ
q = np.floor((sk - u) / delta)[:, None] + np.arange(-search,
search + 1)
centre = u[:, None] + (q + 0.5) * delta                # กึ่งกลาง
ช่วงที่เป็นตัวเลือก
valid = (centre > 0) & (q % 2 == bits[:, None])     # พาริตี้ตรง
กับบิต
order = np.argsort(np.where(valid, np.abs(centre - sk[:,
None]), np.inf), axis=1)
for j in range(order.shape[1]):                        # ลองจาก
ใกล้ไปไกล
    ...
    s_new[:, k] = centre[pend, col]
    # สร้างบล็อกกลับ ปัดเศษ แล้วอ่านแบบเดียวกับผู้รับ
    blk = np.clip(np.rint((U[pend] * s_new[:, None, :]) @
Vt[pend]), 0, 255)
    s_read = np.linalg.svd(blk, compute_uv=False)[:, k]
    read = extract_bit_array(s_read, delta, u[pend])
    pos = ((s_read - u[pend]) / delta) % 1.0
    mg = np.minimum(pos, 1.0 - pos)                    # ระยะถึง
ขอบช่วง (หน่วยของ delta)
    good = ok_cand & (read == bits[pend]) & (mg >= margin)   #
ถูก และห่างขอบ >= delta/4
    sel[pend[good]] = blk[good]
```

4.3.4 คุณภาพภาพขึ้นกับ Δ/b

สมมติ σ ตกตำแหน่งใดก็ได้ในช่วงเท่าๆ กัน ระยะที่ต้องย้าย d มีสองกรณี: พาริตี้ตรงอยู่แล้ว (ครึ่งหนึ่ง) ย้ายไปกลางช่วงเดิม d กระจายใน $[0,\Delta/2)$ ได้ $E[d^2]=\Delta^2/12$ · พาริตี้ไม่ตรง ต้องย้ายไปช่วงข้าง $d=\Delta/2+m,\ m\in[0,\Delta/2)$ ได้ $E[d^2]=\Delta^2/4+\Delta^2/4+\Delta^2/12=7\,\Delta^2/12$ รวมสองกรณี $E[d^2]=\Delta^2/3$ ดังนั้น RMS ของ $d=\Delta/\sqrt{3}=0.577\,\Delta$ และจากข้อ 4.2.5 พิกเซลเปลี่ยน RMS $\approx 0.577\,\Delta/b$

ตัวอย่าง $\Delta/b=4$: พิกเซลเปลี่ยน $\approx 0.577\times 4=2.31$ → PSNR $\approx 20\log_{10}(255/2.31)=40.9$ dB วัดจริงได้ 40.75 dB (การจำลอง 1,000,000 ครั้งได้ RMS $0.5773\,\Delta$ ตรงสูตร) ข้อนี้เป็นค่าประมาณที่เราคิดเอง จึงต้องยืนยันด้วยผลจริง (ตารางที่ 6.1)

4.3.5 รหัสลับ: เลื่อนเส้นแบ่งช่วงของแต่ละบล็อก

ถ้าทุกบล็อกใช้เส้นแบ่งชุดเดียวกัน σ_1/Δ ของทุกบล็อกจะไปกองที่กึ่งกลางช่วง คนนอกตรวจเจอได้ง่าย (ข้อ 6.6) จึงเลื่อนเส้นแบ่งของบล็อกที่ i ไป $u_i\in$ ¿: อ่านบิตด้วย $\lfloor(\sigma-u_i)/\Delta\rfloor \bmod 2$ ค่า $u_1,u_2,\ldots$ สร้างจากตัวสุ่มที่มี seed มาจากรหัสลับ ผู้ส่งและผู้รับที่มีรหัสเดียวกันได้ชุด u_i เดียวกัน seed คำนวณด้วย PBKDF2-HMAC-SHA256 (SHA-256 ซ้ำ 200,000 รอบ) ได้ seed 256 บิต (รุ่นแรกใช้ CRC32 ซึ่งมีแค่ 2^{32} แบบ ลองครบได้ในราว 23 ชั่วโมง ดูบทที่ 7)

โค้ด 4.7 รหัสลับ → ระยะเลื่อนของแต่ละบล็อก (`stego.py`)

```
@lru_cache(maxsize=64)
def _key_seed(key: str) -> int:
    digest = hashlib.pbkdf2_hmac("sha256", key.encode("utf-
8"), KDF_SALT, 200_000)
    return int.from_bytes(digest, "big")

def _dither(n, delta, key):
    if key is None:
        return np.zeros(n)
    return
np.random.default_rng(_key_seed(str(key))).uniform(0.0, 2.0 *
delta, n)
```

4.3.6 ข้อความ → บิต และการตรวจความถูกต้อง

ข้อความเข้ารหัสเป็น UTF-8 (ภาษาไทยใช้ 3 ไบต์ต่อตัวอักษร) แล้วจัดเป็นกรอบ: ความยาว 16 บิต | CRC-16 16 บิต | เนื้อความ CRC คำนวณจากช่องความยาวรวมกับเนื้อความ โดยเริ่มจาก 0xFFFF ผู้รับอ่านหัว 32 บิตก่อน แล้วอ่านเนื้อความตามความยาว ผลมีสามแบบ: **ok** (CRC ตรง), **no_message** (ความยาวเป็น 0 หรือยาวเกินภาพจุได้), **damaged** (CRC ไม่ตรง)

ตัวอย่างความจุ: บล็อก 8×8 มี 2,400 บิต หักหัว 32 บิต เหลือ $\lfloor 2368/8 \rfloor = 296$ ไบต์ ≈ 98 ตัวอักษรไทย

สคริปต์ต่อไปนี้รวมทุกขั้นของหัวข้อ 4.3 บนภาพจริง 249061.jpg (เมทริกซ์ 320×480) ซ่อนข้อความภาษาไทยด้วยรหัสลับ แล้วอ่านกลับสามแบบ ข้อความสั้นจึงแก้แค่บล็อกต้นๆ ของภาพ PSNR ทั้งภาพจึงสูงกว่า 40.7 dB ที่ได้เมื่อซ่อนเต็มความจุ

โค้ด 4.8 **examples/ex6_hide_text.py**

```
from pathlib import Path

import numpy as np

from svdmark.attacks import jpeg
from svdmark.data import load_gray
from svdmark.metrics import psnr
from svdmark.stego import embed_text, extract_text,
text_capacity_bytes

ROOT = Path(__file__).resolve().parents[1]
A = load_gray(ROOT /
"data/images/train/249061.jpg").astype(float)
print(
    "image matrix      :",
    A.shape,
    " capacity",
    text_capacity_bytes(A.shape),
    "bytes",
)

msg = "นัดเจอหน้าตึก ECC 5 โมง"
res = embed_text(A, msg, key="linear2026")  # b = 8, delta =
32, sigma_1
print(
    "PSNR              :",
    round(psnr(A, res.image), 2),
    "dB,",
    res.n_failed,
    "failed blocks",
)

for label, img, key in [
    ("right key        ", res.image, "linear2026"),
    ("wrong key        ", res.image, "linear2025"),
    ("right key, JPEG90", jpeg(res.image, 90), "linear2026"),
]:
    print(label, ":", extract_text(np.asarray(img, float),
key=key))
```

```
Terminal — svd-stego-watermark
$ python examples/ex6_hide_text.py
image matrix      : (320, 480)  capacity 296 bytes
PSNR              : 47.69 dB, 0 failed blocks
right key         : ('นัดเจอหน้าตึก ECC 5 โมง', 'ok')
wrong key         : (None, 'no_message')
right key, JPEG90 : ('นัดเจอหน้าตึก ECC 5 โมง', 'ok')
```

รูปที่ 4.4 ผลรันโค้ด 4.8: รหัสถูกอ่านได้ข้อความครบ (ok) รหัสผิดได้ no_message และหลังบีบอัด JPEG คุณภาพ 90 ยังอ่านได้

4.4 การประยุกต์ที่ 2: ฝังลายน้ำ (Liu & Tan, 2002)

4.4.1 ขั้นตอน

1. SVD ของภาพทั้งภาพ: $A = U\,\Sigma\,V^{T}$
2. บวกลายน้ำเข้าไปใน Σ: $D = \Sigma + \alpha\,W$ (α = ความแรง)
3. SVD ของ D: $D = U_w\,\Sigma_w\,V_w^{T}$
4. ภาพที่ฝังแล้ว: $A_w = U\,\Sigma_w\,V^{T}$ (ใช้ U, V เดิม เปลี่ยนแค่ค่าเอกฐาน)
5. เก็บ key = $(U, V, \Sigma, U_w, V_w, \alpha)$ ไว้สำหรับถอด
6. ถอดจากภาพที่ได้รับ $A^{\prime}$: หา $\Sigma^{\prime}$ ของ $A^{\prime}$ แล้ว $D^{\prime} = U_w\,\Sigma^{\prime}\,V_w^{T}$, $W^{\prime} = (D^{\prime} - \Sigma)/\alpha$

ตัวอย่างที่ 2: ฝังลายน้ำในเมทริกซ์ 2×2 ใช้ A จากตัวอย่างที่ 1 ลายน้ำ W และความแรง $\alpha = 0.5$

$$A = \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix}, W = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}, \alpha = 0.5$$

ขั้นที่ 1 จากตัวอย่างที่ 1 ได้ $\Sigma = diag(6.7082, 2.2361)$ แล้วบวก αW

$$D = \Sigma + \alpha W = \begin{bmatrix} 6.7082 + 0.5 & 0 \\ 0.5 & 2.2361 + 0.5 \end{bmatrix} = \begin{bmatrix} 7.2082 & 0 \\ 0.5 & 2.7361 \end{bmatrix}$$

ขั้นที่ 2 SVD ของ D (เลือกเครื่องหมายให้อ่านง่าย)

$$\Sigma_w = diag(7.2284, 2.7284), U_w = \begin{bmatrix} 0.9967 & 0.0807 \\ 0.0807 & -0.9967 \end{bmatrix}, V_w^T = \begin{bmatrix} 0.9995 & 0.0305 \\ 0.0305 & -0.9995 \end{bmatrix}$$

ขั้นที่ 3 ภาพที่ฝังแล้ว ใช้ U, V เดิมกับค่าเอกฐานใหม่ และตรวจกับข้อ 4.2.5

$$A_w = U \Sigma_w V^T = \begin{bmatrix} 3.4466 & -0.2140 \\ 4.2389 & 5.4591 \end{bmatrix}$$

$$\| A_w - A \|_F = 0.71626 = \sqrt{0.5202^2 + 0.4923^2} = \| \sigma_w - \sigma \| \checkmark$$

ขั้นที่ 4 ถอด: ค่าเอกฐานของ A_w คือ Σ_w จึงได้ D กลับมาพอดี

$$D' = U_w \Sigma_w V_w^T = D \Rightarrow W' = \frac{D' - \Sigma}{0.5} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} = W \checkmark, NCC = 1$$

4.4.2 ช่องโหว่: key เก็บลายน้ำไว้เกือบทั้งหมด

สังเกตว่า U_w "เกือบ" เป็นเมทริกซ์เอกลักษณ์ แต่เอียงไปเป็นมุมเล็กๆ (0.0807) มุมนี้เกิดจากส่วน**นอกเส้นทแยง**ของ αW และอยู่ใน key ไม่ได้อยู่ในภาพ ส่วนที่อยู่ในภาพมีแค่การขยับของ σ ซึ่งมาจากส่วน**บนเส้นทแยง**ของ W ถ้าถอดจาก**ภาพต้นฉบับที่ไม่เคยฝัง** ($\Sigma' = \Sigma$) ส่วนนอกเส้นทแยงจะกลับมาครบ

ตัวอย่าง 3×3 บล็อก B มีค่าเอกฐาน $\sigma = (299.352, 5.937, 3.013)$ ใช้ W = ลายกากบาทในข้อ 4.1.3 และ $\alpha = 4$

$$B = \begin{bmatrix} 100 & 104 & 98 \\ 96 & 100 & 103 \\ 101 & 99 & 97 \end{bmatrix}, D = \Sigma + 4W = \begin{bmatrix} 303.352 & 0 & 4 \\ 0 & 9.937 & 0 \\ 4 & 0 & 7.013 \end{bmatrix}$$

เลข 4 ที่มุมทั้งสอง (ส่วนนอกเส้นทแยง) ทำให้ U_w หมุนในระนาบของแกนที่ 1 กับ 3 เป็นมุมเล็กๆ

$$\theta \approx \frac{4}{303.352 - 7.013} = 0.0135\,\text{rad}$$

ถอดจาก**ภาพต้นฉบับที่ไม่เคยฝัง** ($\Sigma^{¿} = \Sigma$): ช่อง (1,3) ของ

$D^{¿} = U_w \Sigma V_w^T$

$D_{13}^{¿}$ ¿ $(-0.999909)(299.352)(-0.013494) + (0.013494)(3.013)(-0.999909)$

¿ ¿

ส่วนนอกเส้นทแยงกลับมาครบ ขณะที่ส่วนบนเส้นทแยงหายไป ได้ NCC = 0.632 (เท่ากับตัวอย่างในข้อ 4.1.3) ภาพจริงขนาด 320×320 มีเส้นทแยงแค่ 320 ช่องจาก 102,400 ช่อง (0.3%) ภาพที่ไม่เคยฝังจึงได้ NCC สูงถึง 0.94–0.995 (ตารางที่ 6.4) สอดคล้องกับข้อสังเกตของ Zhang & Li (2005)

โค้ด 4.9 ฝัง ถอด และ shift score (`watermark.py`)

```
def embed(A, W, alpha):
    U, s, Vt = svd(A)
    D = np.diag(s) + alpha * W                          # D =
Sigma + alpha W
    Uw, sw, Vwt = svd(D)                                # D =
Uw Sigma_w Vw^T
    Aw = reconstruct(U, sw, Vt)                         # Aw
= U Sigma_w V^T
    return Aw, WatermarkKey(U, Vt, s, Uw, Vwt, alpha, sw)

def extract_from_sigma(s_star, key):
    D_star = reconstruct(key.Uw, s_star, key.Vwt)       # D*
= Uw Sigma* Vw^T
    return (D_star - np.diag(key.S)) / key.alpha        # W*
= (D* - Sigma) / alpha

def shift_from_sigma(s_star, key):
    v, t = s_star - key.S, key.Sw - key.S
    return float(v @ t / (np.linalg.norm(v) *
np.linalg.norm(t)))   # cosine
```

โค้ด 4.10 **examples/ex4_watermark.py** (ตัวอย่างที่ 2 และตัวอย่าง 3×3)

```
import numpy as np

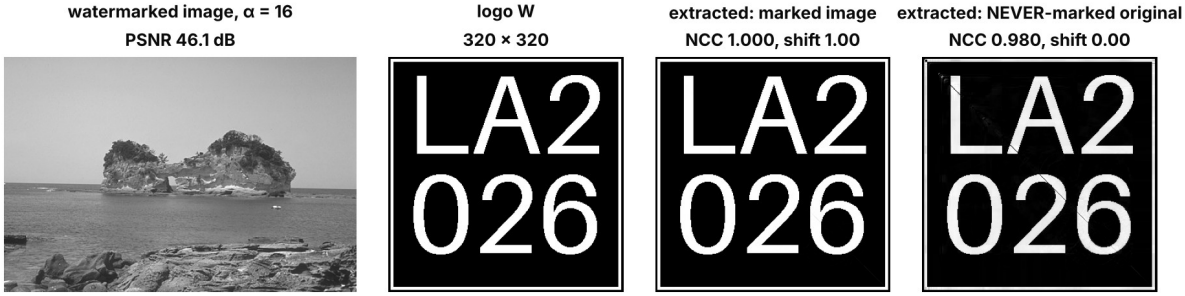
from svdmark.metrics import ncc
from svdmark.watermark import embed, extract, shift_score

A = np.array([[3.0, 0.0], [4.0, 5.0]])
W = np.array([[1.0, 0.0], [1.0, 1.0]])
Aw, key = embed(A, W, alpha=0.5)
print("Sigma_w          =", np.round(key.Sw, 4))
print("A_w              =\n", np.round(Aw, 4))
print(
    "|A_w - A|_F      =",
    round(np.linalg.norm(Aw - A), 5),
    "  |sigma_w - sigma| =",
    round(np.linalg.norm(key.Sw - key.S), 5),
)
print("W* from A_w      =\n", np.round(extract(Aw, key), 4))

B = np.array([[100.0, 104, 98], [96, 100, 103], [101, 99,
97]])
Wx = np.array([[1.0, 0, 1], [0, 1, 0], [1, 0, 1]])
Bw, k3 = embed(B, Wx, alpha=4)
W_fake = extract(B, k3)  # extract from the image that was
NEVER marked
print(
    "3x3 never marked: W*[1,3] =",
    round(W_fake[0, 2], 4),
    " NCC =",
    round(ncc(Wx, W_fake), 3),
    " shift =",
    shift_score(B, k3),
)
print(
    "3x3 marked:       NCC =",
    round(ncc(Wx, extract(Bw, k3)), 3),
    " shift =",
    round(shift_score(Bw, k3), 3),
)
```

```
Terminal — svd-stego-watermark
$ python examples/ex4_watermark.py
Sigma_w          = [7.2284 2.7284]
A_w              =
 [[ 3.4466 -0.214 ]
 [ 4.2389  5.4591]]
|A_w - A|_F      = 0.71626   |sigma_w - sigma| = 0.71626
W* from A_w      =
 [[1. 0.]
 [1. 1.]]
3x3 never marked: W*[1,3] = 0.9996  NCC = 0.632  shift = 0.0
3x3 marked:       NCC = 1.0  shift = 1.0
```

รูปที่ 4.5 ผลรันโค้ด 4.10: ค่า Σ_w, A_w และ $W^{¿}$ ตรงกับตัวอย่างที่ 2 ส่วนภาพ 3×3 ที่**ไม่เคยฝัง**ถอดได้ $W^{¿}_{13}$=0.9996 และ NCC 0.632 แต่ shift score เป็น 0



รูปที่ 4.6 ฝังโลโก้ "LA2026" ด้วย α = 16 แล้วถอด: จากภาพที่ฝัง (ซ้ายที่สาม) และจากภาพต้นฉบับที่**ไม่เคยฝัง** (ขวาสุด) ซึ่งก็ยังได้โลโก้เห็นเส้นทแยงมืดคือส่วนบนเส้นทแยงที่หายไป ตรงตามทฤษฎี แต่ shift score แยกได้ (1.00 กับ 0.00)

4.4.3 shift score (ข้อเสนอของเรา)

แทนที่จะดูว่า $W^{¿}$ หน้าตาเหมือน W ไหม (ซึ่ง key ทำให้เหมือนอยู่แล้ว) เราถามว่า "ค่าเอกฐานของภาพที่ตรวจ ขยับไปทางเดียวกับที่การฝังทำหรือไม่" ด้วย cosine ของสองเวกเตอร์ (ข้อ 4.1.3)

$$\text{shift} = \cos\left(\sigma^{¿} - \sigma, \sigma_w - \sigma\right)$$

ภาพที่ฝังแล้วได้ ≈ 1, ภาพต้นฉบับที่ไม่ถูกแก้ได้ 0 (ไม่มีการขยับ) วิธีนี้ไม่ได้มาจากงานวิจัยที่อ้างอิง

4.5 การประยุกต์ที่ 3: แก้บิตผิดด้วยเมทริกซ์ Hamming(7,4)

CRC บอกได้แค่ว่าข้อความผิด แต่บอกตำแหน่งไม่ได้ บิตผิดแค่บิตเดียวก็ทำให้ข้อความทั้งก้อนถูกปฏิเสธ รหัส Hamming(7,4) เพิ่มบิตตรวจ 3 บิตให้ข้อมูลทุก 4 บิต และแก้บิตผิดได้ 1 บิตในทุกกลุ่ม 7 บิต ทั้งหมดเป็นการคูณเมทริกซ์แบบ mod 2

$$G = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{bmatrix}, H = \begin{bmatrix} 1 & 1 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 1 \end{bmatrix}$$

$G = [I_4 \vee P]$ และ $H = [P^T \vee I_3]$ ทำให้ $GH^T = P + P = 0$ (mod 2) ทุกคอลัมน์ของ H ต่างกันและไม่เป็นศูนย์

ตัวอย่าง เข้ารหัส $d = (1,0,1,1)$: $c = dG$ = แถว 1 + แถว 3 + แถว 4 ของ G (บวกแบบ mod 2)

$$c = (1,0,1,1,\ 1+0+1,\ 1+1+1,\ 0+1+1) = (1,0,1,1,0,1,0)$$

ส่งแล้วบิตที่ 3 พลิก ได้ $r = (1,0,0,1,0,1,0)$ ผู้รับคำนวณ syndrome $s = Hr^T$ ทีละแถวของ H

$$s_1 \quad ¿ 1+0+0+1+0+0+0 = 2 \equiv 0$$
$$s_2 \quad ¿ 1+0+0+1+0+1+0 = 3 \equiv 1 \Rightarrow s = (0,1,1)$$
$$s_3 \quad ¿ 0+0+0+1+0+0+0 = 1$$

แก้: $(0,1,1)$ ตรงกับคอลัมน์ที่ 3 ของ H → บิตที่ 3 ผิด พลิกกลับได้ข้อมูล $(1,0,1,1)$ ✓ ถ้าไม่มีบิตผิด $s=0$ เพราะ $GH^T=0$

ตรวจครบทั้ง 16 ข้อมูลที่เป็นไปได้ × 7 ตำแหน่งบิตผิด ใน `test_all_16_messages_and_all_single_errors_are_corrected`

โค้ด 4.11 Hamming(7,4) (`ecc.py`)

```
P = np.array([[1, 1, 0], [1, 0, 1], [0, 1, 1], [1, 1, 1]],
dtype=np.uint8)
G = np.hstack([np.eye(4, dtype=np.uint8), P])          # 4 x 7
H = np.hstack([P.T, np.eye(3, dtype=np.uint8)])        # 3 x
7,  G H^T = 0 (mod 2)

def encode(bits):
    d = ...reshape(-1, 4)
    return (d @ G % 2).ravel()

def decode(bits):
    r = ...reshape(-1, 7)
    s = (r @ H.T) % 2                                    #
syndrome ของทุกกลุ่ม
    syn = s[:, 0] * 4 + s[:, 1] * 2 + s[:, 2]
    for i in np.flatnonzero(syn):
        r[i, _SYN_TO_POS[int(syn[i])]] ^= 1              #
พลิกบิตที่ syndrome ชี้
    return r[:, :4].ravel(), len(np.flatnonzero(syn))
```

โค้ด 4.12 **examples/ex5_hamming.py**

```
import numpy as np

from svdmark.ecc import G, H, decode, encode

print("G H^T (mod 2) =\n", (G @ H.T) % 2)
d = np.array([1, 0, 1, 1])
c = encode(d)
print("data d        =", d)
print("codeword c    =", c)

r = c.copy()
r[2] ^= 1  # bit 3 flips during transmission
print("received r    =", r)
print("syndrome H r  =", (H @ r) % 2, " = column 3 of H:",
H[:, 2])
d_hat, fixed = decode(r)
print("decoded       =", d_hat, " bits corrected:", fixed)
```

```
Terminal — svd-stego-watermark
$ python examples/ex5_hamming.py
G H^T (mod 2) =
 [[0 0 0]
 [0 0 0]
 [0 0 0]
 [0 0 0]]
data d        = [1 0 1 1]
codeword c    = [1 0 1 1 0 1 0]
received r    = [1 0 0 1 0 1 0]
syndrome H r  = [0 1 1]  = column 3 of H: [0 1 1]
decoded       = [1 0 1 1]  bits corrected: 1
```

รูปที่ 4.7 ผลรันโค้ด 4.12: $GH^T=0$, เข้ารหัส (1,0,1,1) ได้ (1,0,1,1,0,1,0) และ syndrome (0,1,1) ชี้บิตที่ 3 จึงแก้กลับได้

แลกกับ: ความจุเหลือ 4/7 (บล็อก 8×8: 296 → 167 ไบต์) และแก้ได้แค่ 1 บิตต่อกลุ่ม ถ้าบิตผิดหนาแน่น (เบลอ, หมุน, บริเวณที่ถูกเจาะ) จะแก้ไม่ได้

บทที่ 5
ขั้นตอนการประมวลผลที่วางแผนไว้และการเลือกพารามิเตอร์

5.1 แผนการทำงานทั้งหมด

ตารางที่ 5.1 ขั้นตอนการทำงานทั้งหมด (รันขั้น 2 และ 4–8 ด้วย `python experiments/run_all.py` ใช้เวลาราว 1 ชั่วโมงบนเครื่อง 2 คอร์)

ขั้น	งาน	ทฤษฎีที่ใช้	สคริปต์ / ผลลัพธ์
1	สำรวจและ clean ข้อมูล	สถิติพื้นฐาน, เมทริกซ์ภาพ	notebook 00, 01 → `dataset_stats.csv`, `clean_manifest.csv`
2	แบ่งชุด pilot/eval และเก็บค่าเอกฐานทั้งภาพ	SVD, เวกเตอร์ σ	`prepare_data.py` → `image_sets.csv`, `sigma_eval.csv`
3	ทดสอบหน่วย (unit test) เทียบกับตัวอย่างคำนวณมือ	ตัวอย่างที่ 1–3, Hamming	`pytest` (107 เทสต์)
4	ทดลองนำร่องเลือก b, Δ, k, α (ชุด pilot)	ข้อ 4.3.2–4.3.4	`pilot_stego.py`, `pilot_watermark.py`

ขั้น	งาน	ทฤษฎีที่ใช้	สคริปต์ / ผลลัพธ์
5	ทดลองเต็มบนชุด eval 400 ภาพ ทุกการโจมตี	QIM, Liu–Tan	`full_stego.py` (144,000 แถว), `full_watermark.py`
6	ตรวจทฤษฎีกับผลจริง	Frobenius, Weyl, $2Q(\Delta/2s)$	`theory_checks.py`
7	ทดสอบการตรวจจับ (สถิติ R และ machine learning)	cosine/เฟส, รอยบล็อก	`steganalysis.py`, `ml_steganalysis.py`
8	ข้อความทั้งก้อนรอดไหม (Hamming เปิด/ปิด, การโจมตีเพิ่ม), ความแข็งแรงของรหัสลับ	Hamming(7, 4)	`robustness_extra.py`, `key_strength.py`
9	แอปเดโม และการทดสอบกับเพื่อน	ทั้งหมด	`demo/app.py`, `demo/test_cases.json`

5.2 การโจมตีที่ใช้ทดสอบ (กำหนดก่อนดูผล)

ตารางที่ 5.2 การโจมตี (`src/svdmark/attacks.py`) ทุกการโจมตีคืนภาพเป็นจำนวนเต็ม 0–255 เหมือนบันทึกเป็นไฟล์จริง

การโจมตี	ระดับ	เลียนแบบอะไร
สัญญาณรบกวนแบบเกาส์	ส่วนเบี่ยงเบน 1, 2, 4, 8 ระดับสี	สัญญาณรบกวนจากการส่ง/ถ่ายภาพ
บีบอัด JPEG	คุณภาพ 95, 90, 75, 50 (+60 ในการทดลองเพิ่ม)	บันทึกภาพใหม่แอปแชท
เจาะกลางภาพ	5%, 10%, 25% ของพื้นที่	ลบส่วนของภาพ
ย่อแล้วขยายกลับ, เบลอ, median 3×3, หมุน	75%/50%, รัศมี 0.5/1, 3×3, 0.5°/1°	การแก้ภาพทั่วไป (การทดลองเพิ่ม)

5.3 การเลือกพารามิเตอร์ (ชุด pilot 100 ภาพ)

กฎการเลือก (เขียนก่อนดูผล): ต้องได้ PSNR เฉลี่ย ≥ 40 dB และบล็อกที่ล้มเหลว < 0.5% แล้วเลือกตัวที่ BER ภายใต้สัญญาณรบกวน $s=4$ รวมกับ JPEG 75 ต่ำที่สุด

ตารางที่ 5.3 ผลนำร่อง: กฎเลือก $\Delta = 4b$ และ σ_1 ($k=0$) ทุกขนาดบล็อก ค่าตั้งต้นของระบบคือ $b=8, \Delta=32$ ส่วน σ_2 ทนน้อยกว่ามาก (ดีที่สุดที่ $b=8$: BER 4.1% และ 3.7%) และ σ_3, σ_4 ล้มเหลว 3–50% ของบล็อก

b	ความจุ (บิต)	Δ ที่ชนะ	PSNR (dB)	BER noise 4	BER JPEG 75
4	9,600	16	40.73	5.11%	0.67%
8	2,400	32	40.75	0.06%	0.08%
16	600	64	40.76	0.04%	0.02%

ลายน้ำ: PSNR หลังปัดเศษ 79.8 / 60.2 / 49.7 / 44.4 / 40.6 / 35.2 dB ที่ α = 2 / 4 / 8 / 12 / 16 / 24 ค่า $\alpha=16$ เป็นค่าสุดท้ายที่ PSNR เฉลี่ย ≥ 40 dB จึงทดลองเต็มที่ $\alpha \in \{2,4,8,12,16,24\}$

บทที่ 6
ผลลัพธ์

ผลทุกตัวในบทนี้มาจากชุด eval 400 ภาพ (ยกเว้นที่ระบุว่า 100 ภาพ) แหล่งข้อมูลสรุปอยู่ใน `results/findings.md`

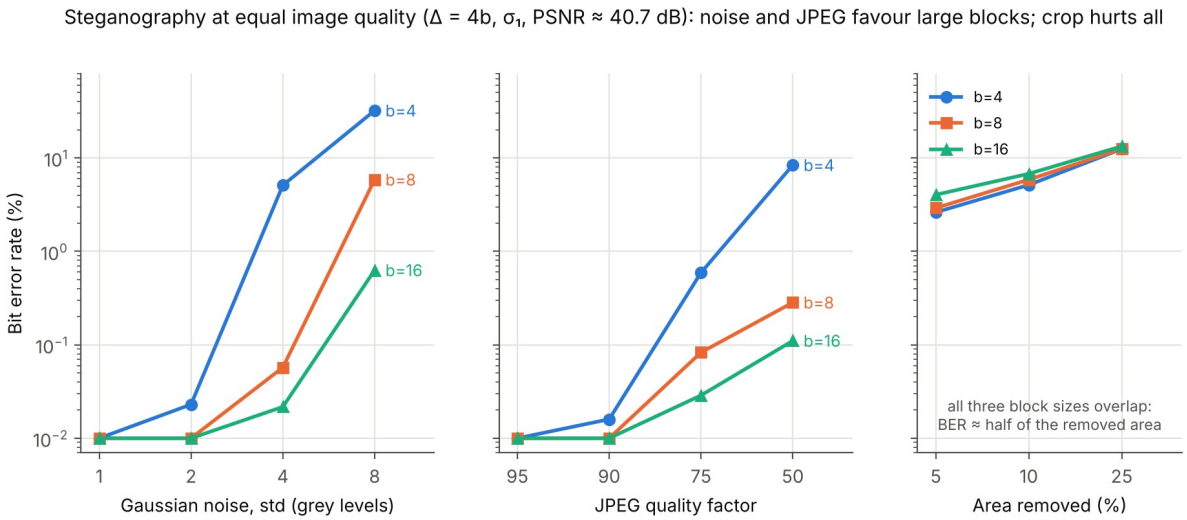
6.1 การซ่อนข้อความ: คุณภาพและความทนทาน

ตารางที่ 6.1 PSNR ขึ้นกับ Δ/b อย่างเดียว ตรงตามข้อ 4.3.4 (ที่ $\Delta=4b$: ภาพ 99.0–99.3% ได้ ≥ 40 dB ภาพที่แย่ที่สุด 39.1 dB)

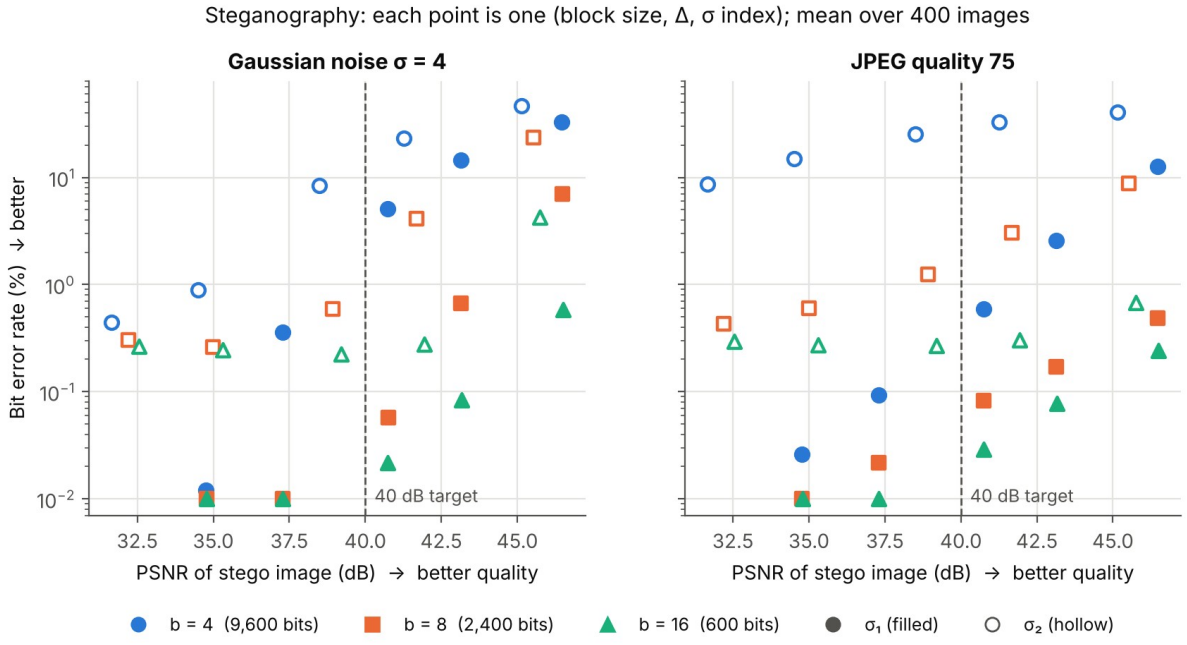
Δ/b	2	3	4	6	8
PSNR เฉลี่ย (dB) ทุกขนาดบล็อก	46.5	43.1	40.7	37.3	34.8

ตารางที่ 6.2 BER (%) เมื่อซ่อนเต็มความจุ ที่ $\Delta=4b$, σ_1 (ไม่ถูกโจมตี: BER = 0 ทุกกรณี) ที่คุณภาพภาพเท่ากัน บล็อกใหญ่ทนกว่ามากแต่จุได้น้อยกว่า 4 และ 16 เท่า · การเจาะภาพทำให้ BER ≈ ครึ่งหนึ่งของพื้นที่ที่ถูกเจาะ เพราะบิตในส่วนนั้นอ่านได้แบบสุ่ม

b	บิต/ภาพ	noise s=4	noise s=8	JPEG 75	JPEG 50	เจาะ 10%	เจาะ 25%
4	9,600	5.10	32.17	0.59	8.38	5.15	12.49
8	2,400	0.06	5.86	0.08	0.28	5.87	12.50
16	600	0.02	0.63	0.03	0.11	6.78	13.36



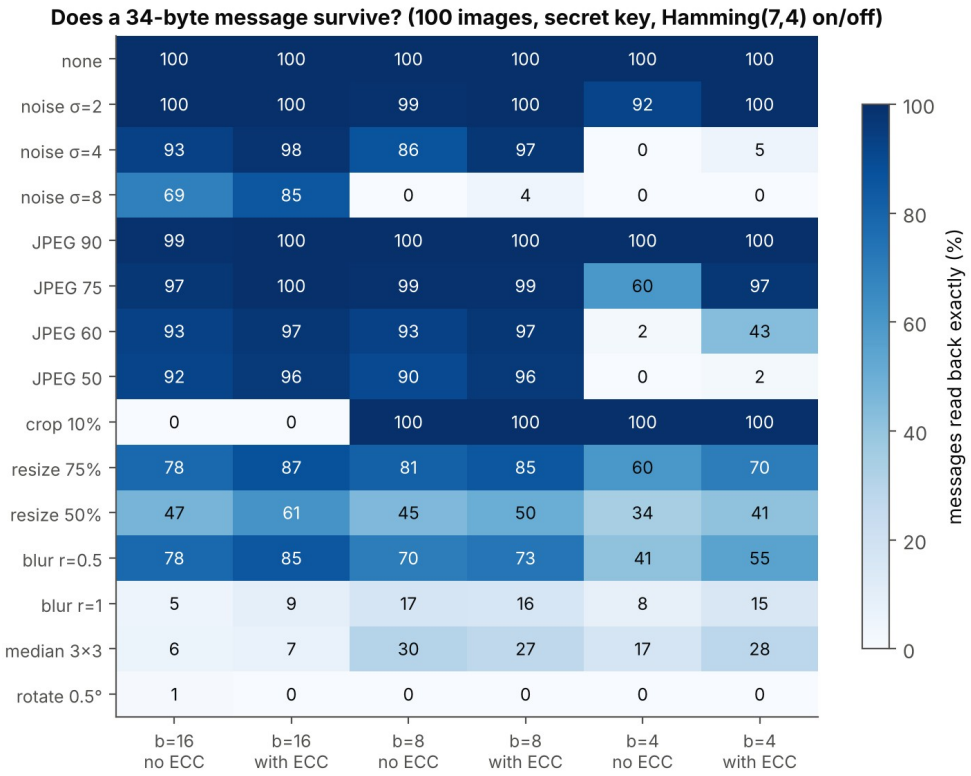
รูปที่ 6.1 BER เทียบระดับการโจมตี ที่คุณภาพภาพเท่ากัน (≈ 40.7 dB)



รูปที่ 6.2 การแลกเปลี่ยนระหว่างคุณภาพภาพ (PSNR) กับความทนทาน (BER) ทุกค่า b, Δ ทั้ง σ_1 (จุดทึบ) และ σ_2 (จุดกลวง)

6.2 ข้อความทั้งก้อนรอดไหม (Hamming เปิด/ปิด)

ข้อความจริงต้องถูกทุกบิต จึงทดลองซ่อนข้อความไทย 34 ไบต์ ("พบกัน 15:00 ตึก 12") พร้อมรหัสลับ ใน 100 ภาพของชุด eval แล้วนับว่าอ่านกลับได้ตรงทุกตัวอักษรกี่ %



รูปที่ 6.3 สัดส่วนข้อความที่อ่านกลับได้ครบ (%) แยกตามขนาดบล็อก การเปิด/ปิด Hamming(7,4) และการโจมตี

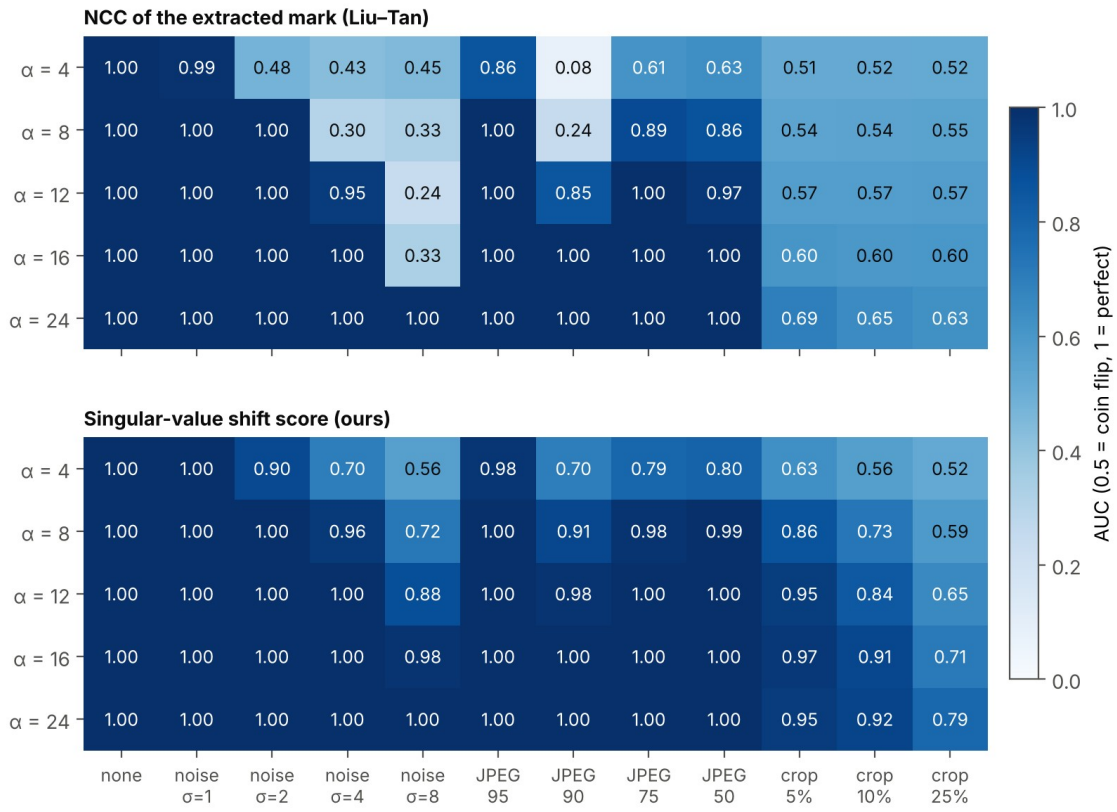
- Hamming ช่วยมากที่สุดเมื่อบิตผิดน้อยแต่ไม่เป็นศูนย์ เช่น บล็อก 4×4 + JPEG 75: 60% → 97% · บล็อก 8×8 + noise 4: 86% → 97%
- การเจาะภาพขึ้นกับ**ตำแหน่ง**ของข้อความ ข้อความสั้นอยู่แถวบนๆ จึงรอดการเจาะตรงกลางที่ $b=4,8$ แต่ที่ $b=16$ ข้อความกินพื้นที่เกือบทั้งภาพจึงเสียหมด

- **การหมุนเพียง 0.5° ทำให้ข้อความหายทุกโหมด** เพราะตารางบล็อกไม่ตรงกับบล็อกเดิม การเบลอรัศมี 1 และ median ก็ทำให้รอดต่ำ (5–30%)

6.3 การฝังลายน้ำ

ตารางที่ 6.4 ผลลายน้ำ (ลายน้ำสุ่ม 0/1 ขนาด 320×320) · ลายน้ำโลโก้ในเดโมมีจุดขาวน้อยกว่า จึงได้ PSNR สูงกว่า (α = 16 ≈ 47.6 dB)

α	**2**	**4**	**8**	**12**	**16**	**24**
PSNR เฉลี่ย (dB)	78.1	59.6	49.3	44.1	40.3	34.9
ภาพที่ได้ ≥ 40 dB (%)	100	100	100	95	58	0
NCC จากภาพที่ฝัง (ไม่โจมตี)	0.997	0.999	1.000	1.000	1.000	1.000
NCC จากภาพเดิมที่**ไม่เคยฝัง**	0.995	0.991	0.981	0.970	0.960	0.939
NCC จากภาพอื่นที่ไม่เกี่ยวข้อง	0.05	0.09	0.17	0.24	0.30	0.38
shift score: ภาพที่ฝัง / ไม่เคยฝัง	0.82 / 0	0.98 / 0	1.00 / 0	1.00 / 0	1.00 / 0	1.00 / 0



รูปที่ 6.4 AUC ของการแยก "ภาพที่ฝังลายน้ำ" กับ "ภาพเดียวกันที่ไม่เคยฝัง" ภายใต้การโจมตีเดียวกัน: NCC (ซ้าย) กับ shift score (ขวา) ค่าต่ำกว่า 0.5 ของ NCC แปลว่าภาพที่ไม่เคยฝังได้ NCC สูงกว่าด้วยซ้ำ

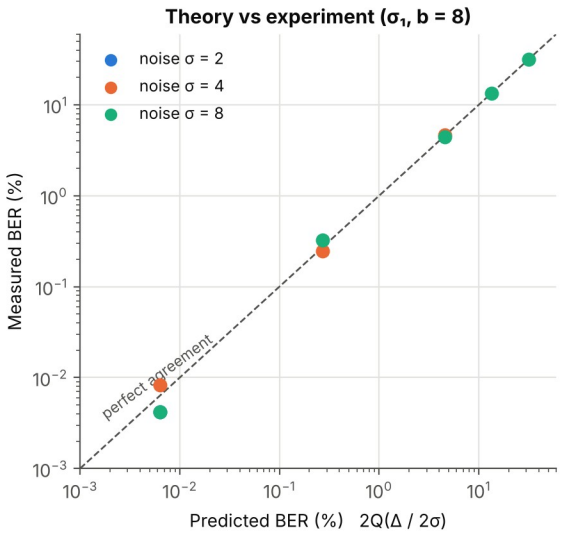
- NCC อย่างเดียวเชื่อไม่ได้: ภายใต้ noise 4 ที่ $\alpha = 8$ ได้ AUC เพียง 0.30 ขณะที่ shift score ได้ 0.96 (JPEG 90: 0.24 เทียบ 0.91)
- ต้องใช้ทั้งสองตัว: NCC แยกภาพที่ฝังกับ**ภาพอื่น**ได้ (AUC 1.0 ยกเว้นการเจาะ) แต่ shift score ทำไม่ได้เสมอ ($\alpha = 2$ หลัง JPEG 50: AUC 0.05)
- $\alpha = 2$ อ่อนเกินไป หลังโจมตีใดๆ ทั้งสองตัววัดใกล้เดาสุ่ม (AUC 0.42–0.64)

- ลายน้ำทนการหมุนเล็กน้อย (หมุน 1°: shift AUC 0.97 / 1.00 ที่ α 8 / 16) แต่ไม่ทนการเบลอ (รัศมี 1: 0.57 / 0.80) และการเจาะ (shift AUC 0.59–0.86 ที่ α = 8)

6.4 ทฤษฎีเทียบกับผลจริง

ตารางที่ 6.5 ทฤษฎีจากบทที่ 4 ตรงกับผลจริงทุกข้อ (`theory_checks.csv`)

การตรวจ	ทฤษฎี	ผลจริง
Frobenius (ข้อ 4.2.5)	$\|A_w - A\|_F = \|\sigma_w -$	ต่างกันมากสุด 6.9×10^{-14} ใน 400 ภาพ ✓
Weyl (ข้อ 4.2.7)	$max_i\|\delta\sigma_i)/\|E\|_2 \le 1$	สูงสุด 0.997 ✓
σ_1 สั่นตามสัญญาณรบกวน	ส่วนเบี่ยงเบน ≈ s	0.997 / 2.001 / 3.997 / 8.017 ที่ s = 1/2/4/8 ✓
BER ภายใต้สัญญาณรบกวน ($b=8$)	$2Q(\Delta/2s)$	$\Delta 16, s4$: 4.55 vs 4.67 · $\Delta 32$, $s8$: 4.55 vs 4.44 · $\Delta 24, s8$: 13.36 vs 13.49 · $\Delta 16, s8$: 31.73 vs 31.60 ✓
กฎการปัดเศษ	$\Delta \ge b$ ปัดเศษแล้วไม่พลิก	ไม่พลิกเลยเมื่อ $\Delta \ge b$ · $\Delta = b/4$ ที่ $b=4$ พลิก 8.4% ✓



รูปที่ 6.5 BER ที่ทำนายด้วย $2Q(\Delta/2s)$ เทียบ BER ที่วัดได้ จุดอยู่บนเส้นทแยงแปลว่าทฤษฎีตรง

6.5 ความแข็งแรงของรหัสลับ

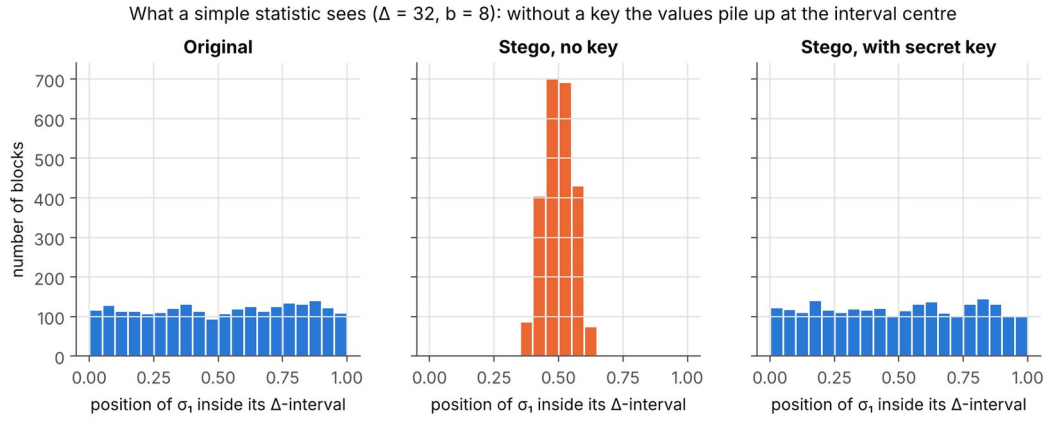
ตารางที่ 6.6 (`key_strength.csv`) รหัสลับซ่อนตำแหน่งของบิต ไม่ได้เข้ารหัสเนื้อข้อความ

วิธีสร้าง seed	ผลการวัด
รุ่นแรก: CRC32(รหัส)	มีแค่ 2^{32} แบบ ลอง seed ได้ ~53,000 แบบ/วินาที → ลองครบใน ~23 ชั่วโมง (เครื่อง 8 คอร์ ~3 ชั่วโมง)

วิธีสร้าง **seed**	ผลการวัด
ปัจจุบัน: PBKDF2-HMAC-SHA256, 200,000 รอบ	seed 256 บิต, ต้องเดารหัสทีละคำ คำละ 0.05–0.07 วินาที · รหัสสุ่ม 8 ตัวพิมพ์เล็ก ~45 ปี (8 คอร์) · แต่คำทั่วไปเช่น "kmitl" ในรายการ 50 คำเจอใน 2.7 วินาที

6.6 ตรวจจับได้ไหมว่าภาพมีข้อมูลซ่อน

ตัวตรวจที่ 1: สถิติ R วัดว่า σ_1/Δ ของทุกบล็อกกองอยู่ที่ตำแหน่งเดียวกันในช่วงหรือไม่ (ความยาวของค่าเฉลี่ยของเวกเตอร์หนึ่งหน่วย $e^{2\pi i\sigma/\Delta}$) แบบไม่ใส่รหัส จับได้ AUC 1.00 แม้ไม่รู้ Δ · ใส่รหัสกับ σ_1: แยกได้แค่ 0.57–0.73 · ใส่รหัสกับ σ_2: แยกได้ 0.87–0.94 เมื่อรู้ Δ (รหัสทำให้ "เรียบเกินธรรมชาติ")

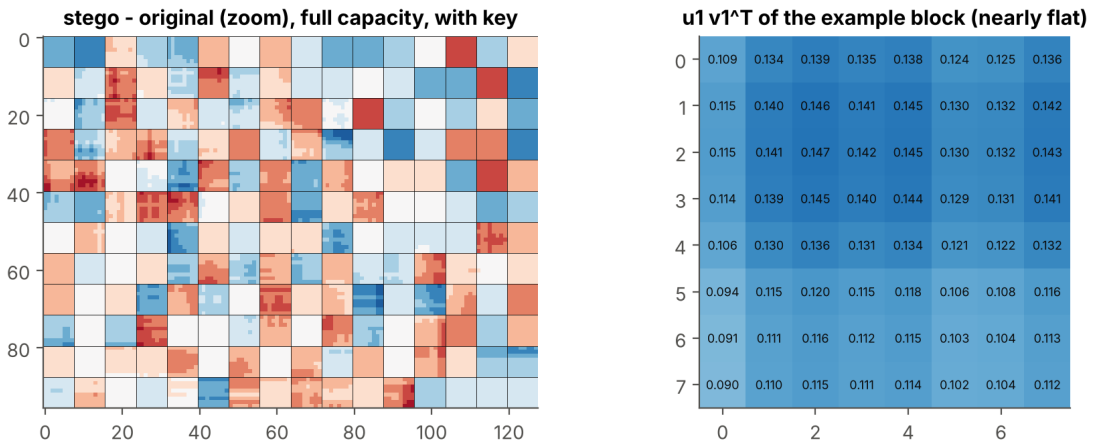


รูปที่ 6.6 ตำแหน่งของ σ_1 ในช่วง (Δ=32, b=8): ภาพปกติกระจายเรียบ, ซ่อนแบบไม่ใส่รหัสกองที่กลางช่วง, ซ่อนแบบใส่รหัสกลับมากระจาย

ตัวตรวจที่ 2: machine learning วัดตัวเลข 11 ค่าจากพิกเซล (รอยต่อที่ขอบบล็อก, สถิติ R, ฮิสโตแกรมผลต่างพิกเซล) ฝึก logistic regression ด้วยชุด pilot แล้วทดสอบกับชุด eval

ตารางที่ 6.7 ตัวตรวจแบบ machine learning (`ml_steganalysis.csv`)

ภาพที่ถูกแก้	AUC	ทายถูก	AUC จากรอยขอบบล็อกอย่างเดียว
ซ่อนเต็มความจุ ไม่ใส่รหัส	1.00	100%	0.91
ซ่อนเต็มความจุ ใส่รหัส ($\Delta = 32$)	0.99	95%	0.91
ใส่รหัส $\Delta = 24$	0.96	89%	0.84
ใส่รหัส $\Delta = 16$	0.86	76%	0.74
ใส่รหัส ข้อความสั้น (16% ของความจุ)	0.79	70%	0.69
ลายน้ำ $\alpha = 8$ / $\alpha = 16$	0.69 / 0.85	63% / 77%	0.55 / 0.59



รูปที่ 6.7 ซ้าย: ผลต่างระหว่างภาพที่ซ่อนข้อความกับภาพเดิม (ขยาย) แต่ละบล็อก 8×8 สว่างขึ้น/มืดลงเกือบเท่ากันทั้งบล็อก เกิด "ขั้น" ที่ขอบขวา: $u_1 v_1^T$ ของบล็อกตัวอย่าง ทุกช่องใกล้ $1/8 = 0.125$

อธิบายด้วยพีชคณิตเชิงเส้น: การเปลี่ยน σ_1 ไป d คือการบวก $d\,u_1 v_1^T$ และในบล็อกที่ค่อนข้างเรียบ $u_1, v_1 \approx 1/\sqrt{8}$ (ข้อ 4.2.6) ดังนั้น $u_1 v_1^T \approx 1/8\,\mathbf{1}\mathbf{1}^T$ เป็นเมทริกซ์ที่ทุกช่องเท่ากัน ทั้งบล็อกจึงเลื่อนความสว่างพร้อมกัน รหัสลับเปลี่ยนแค่ว่า σ_1 ไปตกตรงไหนในช่วง แต่ไม่ได้ลบขั้นนี้ **สรุปตรงๆ: รหัสลับหลบได้แค่สถิติ R แต่ตัวตรวจที่ฝึกแล้วยังจับได้** ลดได้ด้วยการซ่อนน้อยลงหรือใช้ Δ เล็กลง ซึ่งแลกกับความจุหรือความทน

6.7 แอปพลิเคชันเดโมและการทดสอบกับผู้ใช้

แอปเดโม (`python demo/app.py`) เปิดในเบราว์เซอร์ ทำงานบนเครื่องผู้ใช้ทั้งหมด และเรียกโค้ดจริงของโครงงาน มีสามแท็บ: (1) **ซ่อนข้อความ** เลือกรูป พิมพ์ข้อความ (มีแถบแสดงความจุ) ใส่รหัสลับ เลือกโหมด ทนทาน/สมดุล/จุเยอะ เปิดแก้บิตผิดได้ ดาวน์โหลด PNG อ่าน

กลับ และลองโจมตี 19 แบบพร้อมนับบิตผิด (2) **ลายน้ำ** ฝังโลโก้ ปรับ α ถอดจากภาพที่ฝัง/ไม่เคยฝัง/ถูกโจมตี แสดง NCC และ shift score (3) **ทดสอบกับเพื่อน** มีกรณีทดสอบ 20 ข้อพร้อมผลที่ควรได้ (วัดจากภาพจริงก่อนแล้ว) และบันทึกผลลงไฟล์ `demo/demo_log.csv`

ตารางที่ 6.8 สรุปกรณีทดสอบ (รายละเอียดใน `demo/test_cases.json`)

กรณี	สิ่งที่ทดสอบ	ผลที่ควรได้
S1–S4	ซ่อน/อ่านข้อความไทย, รหัสถูก/ผิด/ไม่ใส่	ok · รหัสผิดหรือไม่ใส่ได้ no_message
S5	อ่านภาพที่ไม่เคยซ่อน	ต้องไม่ได้ ok
S6–S7	ข้อความเกินความจุ / เกือบเต็ม	คำเตือนที่อ่านเข้าใจ / ok
S8–S12	JPEG 90, JPEG 50 (โหมดจุเยอะ), noise 2, เจาะภาพ, ภาพเรียบ/ขาวจัด	ตามผลในข้อ 6.1–6.2 รวมกรณีที่ควรพัง
S13–S14	Hamming เปิด/ปิด + JPEG 75, หมุน 0.5°	เปิดแล้วรอดเกือบทุกภาพ · หมุนแล้วพังทุกโหมด
W1–W6	ฝัง/ถอดโลโก้, ถอดจากภาพไม่เคยฝัง, JPEG 75, α = 2, หมุน 1°, เบลอ	shift ≥ 0.8 เมื่อฝังจริง · ภาพไม่เคยฝังได้โลโก้แต่ shift = 0 · เบลอแล้วตัดสินไม่ได้

ผลการทดสอบกับเพื่อน: [เติมหลังทดสอบ: จำนวนผู้ทดสอบ, จำนวนกรณีที่ทดสอบ, สัดส่วนกรณีที่ผลตรงกับที่คาด และข้อสังเกตจากผู้ใช้ จากไฟล์ demo/demo_log.csv]

บทที่ 7
ปัญหาที่พบระหว่างดำเนินโครงงานและวิธีแก้

ตารางที่ 7.1 ปัญหาที่พบและวิธีแก้ ปัญหาส่วนใหญ่พบระหว่างทดสอบหรือทำเดโม และทุกการแก้มีเทสต์รองรับ

ปัญหา	สาเหตุ	วิธีแก้ / ผล
ซ่อนใน σ_2–σ_4 แล้วอ่านผิด	SVD เรียงลำดับใหม่ ค่าที่ตั้งต่ำกว่าตัวถัดไปสลับตำแหน่ง (ข้อ 4.3.2)	ใช้ตรวจซ้ำแบบ closed-loop และเลือก σ_1 เป็นหลัก (σ_1 ล้มเหลว 0 บล็อก)
สูตรอ่านบิตแบบปัดเศษไม่ตรงกับการซ่อน	$round$ กับ $\lfloor \cdot \rfloor$ ให้ช่วงต่างกัน (ข้อ 4.3.1)	ใช้ interval-parity ทั้งซ่อนและอ่าน มีเทสต์กันพลาด
บล็อกขาวจัดพังแม้โดน JPEG 95	พิกเซลชน 255 ถูกตัด ค่า σ ที่อ่านกลับได้บิตถูกแต่ชิดขอบช่วง	เพิ่มระยะเผื่อ $\Delta/4$ ใน closed-loop · BER ที่ b=8 ลด 0.13 → 0.06% (noise 4) และ 0.16 → 0.08% (JPEG 75) ที่ PSNR เท่าเดิม

ปัญหา	สาเหตุ	วิธีแก้ / ผล
ภาพที่ไม่เคยซ่อนอะไรถูกรายงานว่า "ok" (ข้อความว่าง)	บล็อกเรียบอ่านได้บิต 0 ล้วน หัว 32 บิตเป็นศูนย์ = "ยาว 0, CRC 0" ซึ่ง CRC (เริ่มจาก 0) ของข้อความว่างก็เป็น 0 (34% ของภาพในชุด 150 ภาพแรก)	ห้ามข้อความยาว 0, CRC คิดรวมช่องความยาวและเริ่มจาก 0xFFFF · ตรวจ 400×3 กรณีไม่มี ok ผิดอีก
ค่าตั้งต้นของฟังก์ชันไม่ตรงกับค่าที่เลือก	โค้ดช่วงแรกตั้ง $\sigma_2, \Delta=16$	เปลี่ยนเป็น $\sigma_1, b=8, \Delta=32$ และเตือนเมื่อมีบล็อกล้มเหลว
เลือกพารามิเตอร์จากภาพที่ใช้รายงานผล	รุ่นแรกใช้ 30 ภาพนำร่องที่อยู่ในชุด 150 ภาพที่รายงาน	แบ่ง pilot 100 / eval 400 ไม่ซ้ำกันและรันใหม่ทั้งหมด
รหัสลับเดาได้ด้วยการลองครบ	seed จาก CRC32 มีแค่ 2^{32} แบบ (~23 ชั่วโมง)	เปลี่ยนเป็น PBKDF2-HMAC-SHA256 (ข้อ 6.5)

ปัญหา	สาเหตุ	วิธีแก้ / ผล
บิตผิดบิตเดียวทำให้ข้อความทั้งก้อนเสีย	CRC ตรวจได้แต่ซ่อมไม่ได้	เพิ่ม Hamming(7,4) แบบเลือกเปิด (ข้อ 4.5, 6.2) · ลองกระจายบิตทั่วภาพแล้วผลแย่ลงเมื่อโดนเจาะ จึงไม่ใช้
ภาพไม่เคยฝังลายน้ำก็ถอดได้ NCC สูง	key (U_w, V_w) เก็บส่วนนอกเส้นทแยงของลายน้ำ (ข้อ 4.4.2)	เสนอ shift score เป็นตัวตรวจที่สอง และรายงานข้อจำกัดนี้ตรงๆ
ตัวตรวจ machine learning จับได้แม้ใส่รหัส	การเปลี่ยน σ_1 ทำให้เกิดขั้นที่ขอบบล็อก (ข้อ 6.6)	ยังไม่ได้แก้ รายงานเป็นข้อจำกัด และวัดการแลกเปลี่ยนกับ Δ/ความยาวข้อความ
หมุนภาพแล้วข้อความหาย	ตารางบล็อกไม่ตรงกับบล็อกเดิม	ยังไม่ได้แก้ (ต้องมีขั้นจัดภาพให้ตรงก่อนอ่าน) รายงานเป็นข้อจำกัด

7.1 ข้อจำกัดที่ยังเหลืออยู่

- ทดสอบเฉพาะภาพเทา ภาพสีต้องแปลงเป็นเทาก่อน

- การโจมตีเป็นการจำลองด้วยโปรแกรม ยังไม่ได้ทดสอบส่งผ่านแอปแชทจริง
- ตัวตรวจแบบ deep learning (เช่น SRNet) และการลบลายน้ำด้วย AI สร้างภาพ (Zhao et al., 2024) ยังไม่ได้ทดลอง
- shift score เป็นข้อเสนอของเรา ยังไม่ผ่านการตรวจสอบจากงานวิจัยอื่น และเกณฑ์ 0.8/0.3 ในเดโมเป็นค่าคร่าวๆ ใช้ไม่ได้หลังการเบลอ (ค่าติดลบทั้งภาพที่ฝังและไม่ฝัง)

บทที่ 8

Source code และวิธีรัน

Repository: `https://github.com/XoSTka/linear-project`

(Python 3.10+, ไลบรารี: numpy, pandas, pillow, matplotlib, jinja2, pytest)

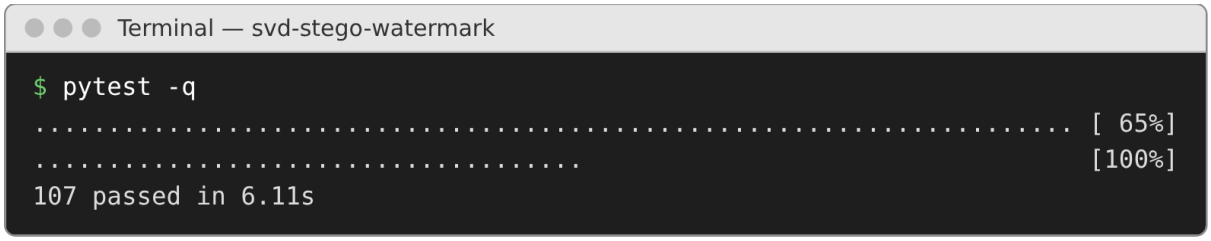
ตารางที่ 8.1 โครงสร้างไฟล์ของ repository

ไฟล์ / โฟลเดอร์	เนื้อหา
`src/svdmark/svd_utils.py`	SVD, การสร้างเมทริกซ์กลับ, SVD 2×2 แบบคำนวณมือ
`src/svdmark/stego.py`	QIM, closed-loop + ระยะเผื่อ, รหัสลับ (PBKDF2), CRC
`src/svdmark/ecc.py`	Hamming(7,4): เมทริกซ์ G, H, syndrome
`src/svdmark/watermark.py`	Liu–Tan และ shift score
`src/svdmark/attacks.py`	สัญญาณรบกวน, JPEG, เจาะ, ย่อขยาย, เบลอ, median, หมุน
`src/svdmark/metrics.py`	MSE, RMS, PSNR, NCC, BER, AUC
`src/svdmark/data.py`	โหลดภาพ, แปลงเทา, ตัดขอบ, ชุด pilot / eval

ไฟล์ / โฟลเดอร์	เนื้อหา
`examples/`	ex1–ex6: สคริปต์ตัวอย่างของบทที่ 4 (โค้ด 4.1–4.12)
`tests/`	107 เทสต์ (คำนวณมือ, ซ่อน-อ่านกลับ, ภาพจริง, Hamming, เดโม)
`experiments/`	run_all.py เรียกทุกขั้นตามลำดับ (ตารางที่ 5.1)
`demo/`	app.py (เว็บเซิร์ฟเวอร์), core.py, static/index.html, test_cases.json
`results/`	CSV, รูป, ตาราง, findings.md, pilot_decisions.md

```
pip install -r requirements.txt
pytest                                  # เทสต์ทั้งหมด
python experiments/run_all.py           # การทดลองทั้งหมด (~1 ชั่วโมง)
python demo/app.py                      # เปิดเดโมที่ http://127.0.0.1:8765
```

โค้ดส่วนที่ใช้ทฤษฎีเวกเตอร์และเมทริกซ์ (SVD, QIM, รหัสลับ, Hamming, ลายน้ำ) แสดงไว้ในบทที่ 4 ตรงกับทฤษฎีแต่ละข้อแล้ว (โค้ด 4.1–4.12) บทนี้จึงสรุปเฉพาะโครงสร้างไฟล์ วิธีรัน และผลการทดสอบ



รูปที่ 8.1 ผลรันเทสต์ทั้งหมดของโครงงาน (107 เทสต์ ผ่านทั้งหมด) รวมเทสต์ที่ตรวจตัวอย่างคำนวณมือในบทที่ 4

บทที่ 9
สรุป

โครงงานนี้ใช้ SVD ซึ่งเป็นเครื่องมือหลักของพีชคณิตเชิงเส้น ทำสองงานจริง คือซ่อนข้อความลับและฝังลายน้ำในภาพ บนข้อมูลภาพ 500 ภาพ (เลือกค่าด้วย 100 ภาพ รายงานผลด้วย 400 ภาพที่ไม่เคยใช้เลือกค่า) ทฤษฎี**เวกเตอร์**ถูกใช้ในความยาวและความตั้งฉากของเวกเตอร์เอกฐาน, cosine (NCC และ shift score) และเวกเตอร์บิตแบบ mod 2 ทฤษฎี**เมทริกซ์**ถูกใช้ในการคูณเมทริกซ์, ค่าลักษณะเฉพาะของ $A^T A$, SVD, Frobenius norm, ทฤษฎีบท Weyl และเมทริกซ์ G, H ของรหัส Hamming

- **ซ่อนข้อความ:** ที่ $b=8, \Delta=32$ ได้ PSNR 40.7 dB ซ่อนได้ 296 ไบต์ อ่านกลับถูก 100% เมื่อไม่ถูกโจมตี และ BER 0.06% / 0.08% ภายใต้ noise 4 / JPEG 75 ทฤษฎี $2Q(\Delta/2s)$ ทำนาย BER ได้แม่นยำ
- **ลายน้ำ:** α = 16 ยังอยู่ที่ขอบ 40 dB, NCC เพียงอย่างเดียวเชื่อไม่ได้ (ภาพไม่เคยฝังได้ NCC 0.94–0.995) และ shift score ที่เราเสนอแยกได้ดีกว่า
- **ข้อจำกัดหลัก:** ตัวตรวจแบบ machine learning จับการซ่อนข้อความได้แม้ใส่รหัส (AUC 0.99 เมื่อซ่อนเต็มความจุ) เพราะรอยที่ขอบบล็อก และข้อความไม่ทนการหมุนภาพ

งานต่อยอด: ลดรอยที่ขอบบล็อก (เช่น ใช้ Δ แปรตามความเรียบของบล็อก), เพิ่มการจัดภาพให้ตรงก่อนอ่าน เพื่อให้ทนการหมุน, ใช้รหัสแก้บิตผิดที่แรงกว่า (เช่น BCH), ทดสอบกับตัวตรวจ deep learning และการส่งผ่านแอปแชทจริง

บทที่ 10
แหล่งอ้างอิง

1. P. Arbeláez, M. Maire, C. Fowlkes, J. Malik. "Contour Detection and Hierarchical Image Segmentation." *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(5):898–916, 2011. ชุดข้อมูล: https://www2.eecs.berkeley.edu/Research/Projects/CS/vision/grouping/resources.html
2. R. Liu, T. Tan. "An SVD-based watermarking scheme for protecting rightful ownership." *IEEE Transactions on Multimedia*, 4:121–128, 2002.
3. X.-P. Zhang, K. Li. "Comments on 'An SVD-based watermarking scheme for protecting rightful ownership'." *IEEE Transactions on Multimedia*, 7:593–594, 2005.
4. B. Chen, G. W. Wornell. "Quantization index modulation: A class of provably good methods for digital watermarking and information embedding." *IEEE Transactions on Information Theory*, 47(4):1423–1443, 2001.

5. R. W. Hamming. "Error detecting and error correcting codes." *Bell System Technical Journal*, 29(2):147–160, 1950.
6. K. Moriarty, B. Kaliski, A. Rusch. "PKCS #5: Password-Based Cryptography Specification Version 2.1." RFC 8018, IETF, 2017.
7. M. Boroumand, M. Chen, J. Fridrich. "Deep Residual Network for Steganalysis of Digital Images." *IEEE Transactions on Information Forensics and Security*, 14(5), 2019.
8. X. Zhao et al. "Invisible Image Watermarks Are Provably Removable Using Generative AI." *NeurIPS*, 2024.
9. R. A. Horn, C. R. Johnson. *Matrix Analysis*, 2nd ed. Cambridge University Press, 2013. (ทฤษฎีบท Weyl, SVD)
10. G. Strang. *Introduction to Linear Algebra*, 5th ed. Wellesley-Cambridge Press, 2016. (เวกเตอร์, ค่าลักษณะเฉพาะ, SVD)
11. ITU-R Recommendation BT.601. "Studio encoding parameters of digital television." (สูตรแปลงภาพสีเป็นภาพเทา)

12. NumPy (Harris et al., *Nature* 585:357–362, 2020) และ Pillow — ไลบรารีที่ใช้คำนวณ SVD และอ่าน/เขียนภาพ

ผลการทดลองทั้งหมด: ไฟล์ใน `results/` ของ repository (สร้างใหม่ได้ด้วย `python experiments/run_all.py`)